종합실습4_제출리포트_작성틀

종합실습 4 - E-Commerce 데이터 요청 및 SQL 튜닝 리포트

- 소속: 판교 5반
- 성명: 홍은비
- DBMS: PostgreSQL _____
- 실행환경: Local PC / DBeaver _____
- 측정 명령: `EXPLAIN (ANALYZE, BUFFERS, SUMMARY)`

> 시드 데이터는 `random()`으로 생성되므로 실행 결과와 시간은 환경마다 달라질 수 있다. 절대시간뿐 아니라 Scan/Join 방식, 반복 횟수, 예상·실제 행 수, Buffers 변화를 함께 비교한다.

1. 실습 목적

구매·재무 등 다른 실무 팀에서 요청한 E-Commerce 자료를 정확한 기준으로 산출하고, 같은 결과를 더 효율적으로 반환하도록 SQL과 인덱스를 개선한다. Q1~Q10은 튜닝 전후 쿼리와 실행계획을 비교하고, 별도로 PostgreSQL의 세 가지 물리 조인과 Materialized View 활용 방법을 검토한다.

2. 공통 데이터 기준

항목	정의
실매출 주문	`order_status IN ('paid','shipped','delivered')`
주문 매출	`SUM(order_items.line_total)`
AOV	실매출 합계 ÷ 실매출 주문 수
Q1 기간	실행시점 기준 최근 1개월
Q3 기간	실행시점 기준 최근 90일
Q6 재구매	첫 구매 후 30일 이내 추가 실구매
Q10 대상	전체 3,000명 누적매출 상위 1%인 30명
Q10 표시금액	선정된 30명의 최근 60일 실매출

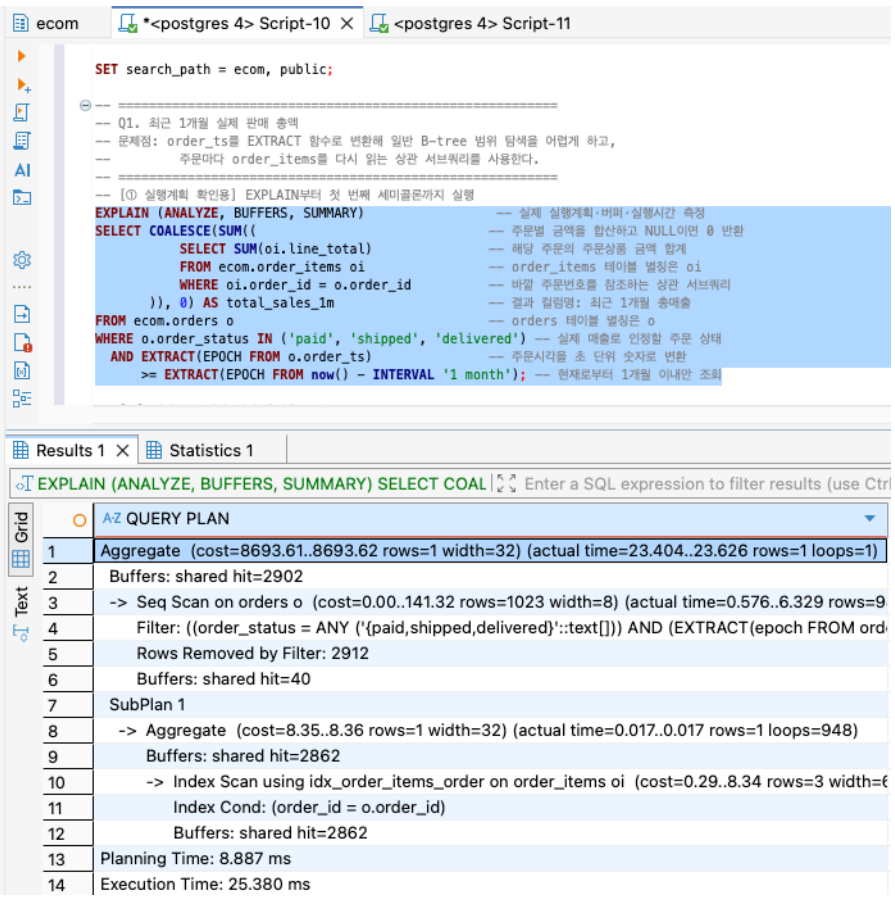
3. Q1 - 최근 1개월 실제 판매 총액

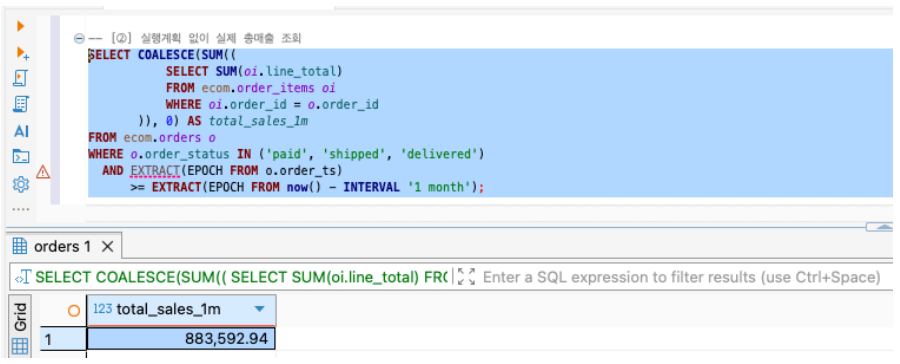
요청사항

`paid`, `shipped`, `delivered` 상태이면서 `now() - interval '1 month'` 이후인 주문의 `line_total` 합계를 제공한다.

튜닝 전 쿼리·결과 화면

Q1 튜닝 전





느린 이유

`order_ts`를 함수로 변환하면 일반 날짜 인덱스의 범위탐색이 어려워질 수 있다. 또한 주문마다 주문상품 합계를 상관 서브쿼리로 반복 조회한다.

개선 방법

날짜 컬럼을 그대로 둔 SARG 범위 조건을 사용하고, 주문과 주문상품을 한 번 조인해 전체 합계를 계산한다. 실매출 상태만 포함하는 부분 인덱스로 대상 주문 탐색 범위를 줄인다.

튜닝 후 쿼리·결과 화면

Q1 튜닝 후

```
SET search_path = ecom, public;

-- Q1. 최근 1개월 SARG 범위 조건 + 한 번의 조인/집계
-- 튜닝: 주문별 상관 서브쿼리 대신 orders와 order_items를 한 번 조인해 합산한다.
-- [①] 실행계획 확인용] 날짜 범위 및 인덱스 사용 여부 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                    -- 튜닝 후 실행계획과 성능 측정
SELECT COALESCE(ROUND(SUM(oi.line_total), 2), 0) AS total_sales_1m -- 총매출, NULL이면 0
FROM ecom.orders o                                     -- 주문 테이블
JOIN ecom.order_items oi ON oi.order_id = o.order_id -- 주문상품을 한 번 조인
WHERE o.order_status IN ('paid', 'shipped', 'delivered') -- 실매출 주문만 포함
  AND o.order_ts >= now() - INTERVAL '1 month';     -- 달력을 변환하지 않는 인덱스 친화적 조건
```

Results 1 | Statistics 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT COAL

	QUERY PLAN
1	Aggregate (cost=292.69..292.70 rows=1 width=32) (actual time=4.852..4.853 rows=1 loops=1)
2	Buffers: shared hit=101 read=8
3	-> Hash Join (cost=55.65..286.56 rows=2449 width=6) (actual time=1.941..4.531 rows=2563 loops=1)
4	Hash Cond: (oi.order_id = o.order_id)
5	Buffers: shared hit=101 read=8
6	-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.006..1.128 rows=10367 loops=1)
7	Buffers: shared hit=100
8	-> Hash (cost=44.25..44.25 rows=912 width=8) (actual time=1.917..1.917 rows=947 loops=1)
9	Buckets: 1024 Batches: 1 Memory Usage: 45kB
10	Buffers: shared hit=1 read=8
11	-> Index Only Scan using idx_orders_sales_ts_order on orders o (cost=0.28..44.25 rows=912 width=8) (actual time=0.315..1.600 rows=947 loops=1)
12	Index Cond: (order_ts >= (now() - '1 mon'::interval))
13	Heap Fetches: 0
14	Buffers: shared hit=1 read=8
15	Planning:
16	Buffers: shared hit=221 read=7
17	Planning Time: 3.082 ms
18	Execution Time: 4.903 ms

```
SELECT COALESCE(ROUND(SUM(oi.line_total), 2), 0) AS total_sales_1m -- [②] 실제 총매출 확인
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts >= now() - INTERVAL '1 month';
```

orders(+) 1

SELECT COALESCE(ROUND(SUM(oi.line_total), 2), 0) AS

	total_sales_1m
1	883,029.03

구분	Scan/Join 방식	Buffers	Execution Time	결과 금액
튜닝 전	_____	_____	_____ ms	_____
튜닝 후	_____	_____	_____ ms	_____

결과 해석: _____

4. Q2 - 월별 주문 수·매출·AOV

요청사항

월별 실매출 주문 수, 총매출, 주문당 평균 금액을 제공한다.

Q2 튜닝 전

```
-- ==========================================================
-- Q2. 월별 주문 수, 매출, AOV
-- 문제점: 상세 행을 먼저 모두 조인한 뒤 COUNT(DISTINCT)로 중복을 제거한다.
-- ==========================================================
-- [①] 실행계획 확인용] 월별 집계의 Scan/Join과 실행시간 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                    -- 실제 실행계획과 성능 측정
SELECT date_trunc('month', o.order_ts)::date AS sales_month, -- 주문시각을 월 단위로 묶음
       COUNT(DISTINCT o.order_id) AS order_count,  -- 상세행 중복을 제거한 주문 수
       ROUND(SUM(oi.line_total), 2) AS revenue,    -- 월별 매출 합계, 소수점 둘째 자리
       ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov -- 주문당 평균금액
FROM ecom.orders o                                     -- 주문 테이블
JOIN ecom.order_items oi ON oi.order_id = o.order_id -- 주문번호로 주문상품 연결
WHERE o.order_status IN ('paid', 'shipped', 'delivered') -- 실매출 주문만 포함
GROUP BY 1                                             -- SELECT 첫 번째 컬럼인 월로 그룹화
ORDER BY 1;                                            -- 월 오름차순 정렬
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT date_

	QUERY PLAN
1	GroupAggregate (cost=939.49..1113.93 rows=3068 width=76) (actual time=18.581..19.750 rows=6 loops=1)
2	Group Key: ((date_trunc('month'::text, o.order_ts))::date)
3	Buffers: shared hit=140
4	-> Sort (cost=939.49..960.09 rows=8240 width=18) (actual time=18.235..18.529 rows=8230 loops=1)
5	Sort Key: ((date_trunc('month'::text, o.order_ts))::date), o.order_id
6	Sort Method: quicksort Memory: 706kB
7	Buffers: shared hit=140
8	-> Hash Join (cost=131.43..403.54 rows=8240 width=18) (actual time=5.049..10.315 rows=8230 loops=1)
9	Hash Cond: (oi.order_id = o.order_id)
10	Buffers: shared hit=140
11	-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.021..1.583 rows=10367 loops=1)
12	Buffers: shared hit=100
13	-> Hash (cost=93.08..93.08 rows=3068 width=16) (actual time=4.279..4.280 rows=3068 loops=1)
14	Buckets: 4096 Batches: 1 Memory Usage: 176kB
15	Buffers: shared hit=40
16	-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=16) (actual time=0.032..1.630 rows=3068 loops=1)
17	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
18	Rows Removed by Filter: 792
19	Buffers: shared hit=40
20	Planning:
21	Buffers: shared hit=12
22	Planning Time: 2.560 ms
23	Execution Time: 19.988 ms

```
SELECT date_trunc('month', o.order_ts)::date AS sales_month, -- [②] 실제 월별 결과 조회
       COUNT(DISTINCT o.order_id) AS order_count,
       ROUND(SUM(oi.line_total), 2) AS revenue,
       ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY 1
ORDER BY 1;
```

orders(+) 1

SELECT date_trunc('month', o.order_ts)::date AS sales_m

	sales_month	order_count	revenue	aov
1	2026-04-01	267	228,521.46	855.89
2	2026-05-01	673	551,927.62	820.1
3	2026-06-01	794	717,531.35	903.69
4	2026-07-01	892	829,725.16	930.19
5	2026-08-01	439	390,509.57	889.54
6	2026-09-01	3	812.51	270.84

느린 이유와 개선 방법

상세 주문상품을 먼저 조인한 상태에서는 주문이 여러 행으로 증가하므로 `COUNT(DISTINCT order_id)`가 필요하다. 튜닝 후에는 주문별 금액을 먼저 집계한 다음 월별로 다시 집계해 중복 제거 비용을 줄인다.

Q2 튜닝 후

```sql
-- Q2. 주문별 매출을 먼저 계산해 COUNT(DISTINCT)를 제거
-- 튜닝: 월별 집계 시 주문이 이미 한 행이므로 COUNT(DISTINCT) 대신 COUNT(*)를 사용한다.
-- [① 실행계획 확인용] 주문별 선집계 후 월별 집계 계획 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH order_totals AS (                          -- 주문별 금액을 먼저 만드는 CTE
    SELECT o.order_id,
           date_trunc('month', o.order_ts)::date AS sales_month,
           SUM(oi.line_total) AS order_total     -- 주문상품을 주문 단위로 합산
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.order_ts
)
SELECT sales_month,
       COUNT(*) AS order_count,                  -- 주문당 한 행이므로 단순 COUNT
       ROUND(SUM(order_total), 2) AS revenue,
       ROUND(AVG(order_total), 2) AS aov
FROM order_totals
GROUP BY sales_month
ORDER BY sales_month;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH order_tc

	QUERY PLAN
1	Sort (cost=522.57..523.07 rows=200 width=76) (actual time=10.499..10.501 rows=6 loops=1)
2	Sort Key: ((date_trunc('month'::text, o.order_ts))::date)
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=143
5	-> HashAggregate (cost=510.92..514.92 rows=200 width=76) (actual time=10.483..10.488 rows=6 loops=1)
6	Group Key: (date_trunc('month'::text, o.order_ts))::date
7	Batches: 1 Memory Usage: 40kB
8	Buffers: shared hit=140
9	-> HashAggregate (cost=403.54..457.23 rows=3068 width=52) (actual time=8.034..9.932 rows=3068 loops=1)
10	Group Key: o.order_id
11	Batches: 1 Memory Usage: 1393kB
12	Buffers: shared hit=140
13	-> Hash Join (cost=131.43..362.34 rows=8240 width=22) (actual time=1.735..5.462 rows=8230 loops=1)
14	Hash Cond: (oi.order_id = o.order_id)
15	Buffers: shared hit=140
16	-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.014..1.409 rows=10367 loops=1)
17	Buffers: shared hit=100
18	-> Hash (cost=93.08..93.08 rows=3068 width=16) (actual time=1.691..1.691 rows=3068 loops=1)
19	Buckets: 4096 Batches: 1 Memory Usage: 176kB
20	Buffers: shared hit=40
21	-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=16) (actual time=0.009..1.213 rows=3068 loops=1)
22	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
23	Rows Removed by Filter: 792
24	Buffers: shared hit=40
25	Planning:
26	Buffers: shared hit=34
27	Planning Time: 1.227 ms
28	Execution Time: 10.664 ms

```sql
WITH order_totals AS (                          -- [②] 실제 월별 결과 조회
    SELECT o.order_id, date_trunc('month', o.order_ts)::date AS sales_month,
           SUM(oi.line_total) AS order_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.order_ts
)
SELECT sales_month, COUNT(*) AS order_count,
       ROUND(SUM(order_total), 2) AS revenue,
       ROUND(AVG(order_total), 2) AS aov
FROM order_totals
GROUP BY sales_month
ORDER BY sales_month;
```

Results 1

WITH order_totals AS (SELECT o.order_id, date_trunc('m

	sales_month	order_count	revenue	aov
1	2026-04-01	267	228,521.46	855.89
2	2026-05-01	673	551,927.62	820.1
3	2026-06-01	794	717,531.35	903.69
4	2026-07-01	892	829,725.16	930.19
5	2026-08-01	439	390,509.57	889.54
6	2026-09-01	3	812.51	270.84

구분	집계 방식	Buffers	Execution Time	전후 결과 동일 여부
튜닝 전	상세행 + `COUNT(DISTINCT)`	_____	_____ ms	-
튜닝 후	주문별 선집계 + `COUNT(*)`	_____	_____ ms	_____

결과 해석: _____

5. Q3 - 최근 90일 카테고리 매출 Top 10

요청사항

최근 90일 실매출을 카테고리별로 집계해 상위 10개를 제공한다.

Q3 튜닝 전

```sql
-- ==========================================
-- Q3. 최근 90일 카테고리 Top 10
-- 문제점: 비-SARG 날짜 조건을 사용하고 상세 데이터를 그대로 집계한다.
-- ==========================================
-- [① 실행계획 확인용] 최근 90일 필터와 테이블 조인 계획 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)             -- 실제 실행계획과 성능 측정
SELECT c.category_id,
       c.category_name,
       ROUND(SUM(oi.line_total), 2) AS revenue_90d -- 카테고리별 최근 90일 매출
FROM ecom.orders o                              -- 주문에서 시작
JOIN ecom.order_items oi ON oi.order_id = o.order_id -- 주문상품 연결
JOIN ecom.products p ON p.product_id = oi.product_id -- 상품 연결
JOIN ecom.categories c ON c.category_id = p.category_id -- 카테고리 연결
WHERE o.order_status IN ('paid', 'shipped', 'delivered') -- 실매출 주문만 포함
  AND o.order_ts::date >= CURRENT_DATE - 90     -- timestamp를 date로 변환해 최근 90일 필터
GROUP BY c.category_id, c.category_name         -- 카테고리별로 묶어 합산
ORDER BY revenue_90d DESC, c.category_id        -- 매출이 큰 순서로 정렬
LIMIT 10;                                       -- 상위 10개만 표시
```

Results 1 | Statistics 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.cate

	QUERY PLAN
1	Limit (cost=497.38..497.41 rows=10 width=72) (actual time=11.220..11.225 rows=10 loops=1)
2	Buffers: shared hit=148
3	-> Sort (cost=497.38..500.06 rows=1070 width=72) (actual time=11.219..11.222 rows=10 loops=1)
4	Sort Key: (round(sum(oi.line_total), 2)) DESC, c.category_id
5	Sort Method: quicksort Memory: 25kB
6	Buffers: shared hit=148
7	-> HashAggregate (cost=458.21..474.26 rows=1070 width=72) (actual time=10.966..10.979 rows=10 loops=1)
8	Group Key: c.category_id
9	Batches: 1 Memory Usage: 73kB
10	Buffers: shared hit=148
11	-> Hash Join (cost=199.04..444.47 rows=2748 width=46) (actual time=3.828..9.543 rows=6662 loops=1)
12	Hash Cond: (p.category_id = c.category_id)
13	Buffers: shared hit=148
14	-> Hash Join (cost=164.96..403.15 rows=2748 width=14) (actual time=3.311..7.831 rows=6662 loops=1)
15	Hash Cond: (oi.product_id = p.product_id)
16	Buffers: shared hit=147
17	-> Hash Join (cost=144.46..375.38 rows=2748 width=14) (actual time=2.841..6.120 rows=6662 loops=1)
18	Hash Cond: (oi.order_id = o.order_id)
19	Buffers: shared hit=140
20	-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=22) (actual time=0.015..1.308 rows=10367 loops=1)
21	Buffers: shared hit=100
22	-> Hash (cost=131.68..131.68 rows=1023 width=8) (actual time=2.502..2.502 rows=2483 loops=1)
23	Buckets: 4096 (originally 1024) Batches: 1 (originally 1) Memory Usage: 129kB
24	Buffers: shared hit=40

```sql
SELECT c.category_id,                           -- [②] 실제 Top 10 결과 조회
       c.category_name,
       ROUND(SUM(oi.line_total), 2) AS revenue_90d
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
JOIN ecom.products p ON p.product_id = oi.product_id
JOIN ecom.categories c ON c.category_id = p.category_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
  AND o.order_ts::date >= CURRENT_DATE - 90
GROUP BY c.category_id, c.category_name
ORDER BY revenue_90d DESC, c.category_id
LIMIT 10;

-- ==========================================
-- 04. 제품별 누적매출 RANK() Top 20
```

categories 1

SELECT c.category_id, c.category_name, ROUND(SUM(oi.

	category_id	category_name	revenue_90d
1	13	Outdoor	758,672.65
2	2	Phones	541,480.74
3	6	Appliances	138,788.46
4	14	Fitness	125,711.82
5	11	Shoes	124,829.38
6	7	Cookware	117,640.32
7	4	Audio	112,653.77
8	3	Laptops	109,500.6
9	9	Men	99,046.69
10	10	Women	96,304.36

```
Results 1 | Statistics 1
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.cate
QUERY PLAN
                    -> Seq Scan on order_items oi  (cost=0.00..203.67 rows=10367 width=22) (actual time=0.015..1.308 rows=10367 loops=1)
                       Buffers: shared hit=100
                    -> Hash  (cost=131.68..131.68 rows=1023 width=8) (actual time=2.502..2.502 rows=2483 loops=1)
                       Buckets: 4096 (originally 1024)  Batches: 1 (originally 1)  Memory Usage: 129kB
                       Buffers: shared hit=40
                       -> Seq Scan on orders o  (cost=0.00..131.68 rows=1023 width=8) (actual time=0.023..1.672 rows=2483 loops=1)
                          Filter: ((order_status = ANY ('{paid,shipped,delivered}'::text[])) AND ((order_ts)::date >= (CURRENT_DATE - 90)))
                          Rows Removed by Filter: 1377
                          Buffers: shared hit=40
                 -> Hash  (cost=13.00..13.00 rows=600 width=16) (actual time=0.449..0.450 rows=600 loops=1)
                    Buckets: 1024  Batches: 1  Memory Usage: 37kB
                    Buffers: shared hit=7
                    -> Seq Scan on products p  (cost=0.00..13.00 rows=600 width=16) (actual time=0.006..0.368 rows=600 loops=1)
                       Buffers: shared hit=7
              -> Hash  (cost=20.70..20.70 rows=1070 width=40) (actual time=0.276..0.276 rows=14 loops=1)
                 Buckets: 2048  Batches: 1  Memory Usage: 17kB
                 Buffers: shared hit=1
                 -> Seq Scan on categories c  (cost=0.00..20.70 rows=1070 width=40) (actual time=0.256..0.257 rows=14 loops=1)
                    Buffers: shared hit=1
Planning:
 Buffers: shared hit=27
Planning Time: 6.885 ms
Execution Time: 11.863 ms
```

느린 이유와 개선 방법

비-SARG 날짜 조건은 인덱스 범위 탐색을 방해할 수 있다. 튜닝 후에는 최근 실매출 주문을 먼저 축소하고 `order_items → products → categories` 순서로 연결한다.

Q3 튜닝 후

```sql
-- Q3. 기간/상태로 orders를 먼저 축소하고 SARG 범위 조건 사용
-- 튜닝: 최근 90일 실매출 주문만 먼저 골라 조인해야 할 행 수를 줄인다.
-- [① 실행계획 확인용] 최근 주문 선필터와 조인 계획 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH recent_sales AS MATERIALIZED (                -- 최근 주문을 먼저 계산해 저장하는 CTE
    SELECT order_id
    FROM ecom.orders
    WHERE order_status IN ('paid', 'shipped', 'delivered')
      AND order_ts >= CURRENT_DATE - 90            -- 컬럼을 변환하지 않는 최근 90일 조건
)
SELECT c.category_id,
       c.category_name,
       ROUND(SUM(oi.line_total), 2) AS revenue_90d
FROM recent_sales rs
JOIN ecom.order_items oi ON oi.order_id = rs.order_id
JOIN ecom.products p ON p.product_id = oi.product_id
JOIN ecom.categories c ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY revenue_90d DESC, c.category_id
LIMIT 10;
```

```
Results 1
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH recent_
QUERY PLAN
Limit  (cost=659.69..659.71 rows=10 width=72) (actual time=11.383..11.387 rows=10 loops=1)
  Buffers: shared hit=120 read=9
  CTE recent_sales
    -> Index Only Scan using idx_orders_sales_ts_order on orders  (cost=0.28..111.02 rows=2442 width=8) (actual time=0.413..1.477 rows=2483 loops=1)
       Index Cond: (order_ts >= (CURRENT_DATE - 90))
       Heap Fetches: 0
       Buffers: shared hit=12 read=9
  -> Sort  (cost=548.67..551.34 rows=1070 width=72) (actual time=11.382..11.385 rows=10 loops=1)
     Sort Key: (round(sum(oi.line_total), 2)) DESC, c.category_id
     Sort Method: quicksort  Memory: 25kB
     Buffers: shared hit=120 read=9
     -> HashAggregate  (cost=509.50..525.55 rows=1070 width=72) (actual time=11.354..11.369 rows=10 loops=1)
        Group Key: c.category_id
        Batches: 1  Memory Usage: 73kB
        Buffers: shared hit=120 read=9
        -> Hash Join  (cost=133.94..476.70 rows=6559 width=46) (actual time=2.623..9.671 rows=6662 loops=1)
           Hash Cond: (p.category_id = c.category_id)
           Buffers: shared hit=120 read=9
           -> Hash Join  (cost=99.87..425.34 rows=6559 width=14) (actual time=2.587..8.147 rows=6662 loops=1)
              Hash Cond: (oi.product_id = p.product_id)
              Buffers: shared hit=119 read=9
              -> Hash Join  (cost=79.37..387.50 rows=6559 width=14) (actual time=2.372..6.349 rows=6662 loops=1)
                 Hash Cond: (oi.order_id = rs.order_id)
                 Buffers: shared hit=112 read=9
                 -> Seq Scan on order_items oi  (cost=0.00..203.67 rows=10367 width=22) (actual time=0.016..1.415 rows=10367 loops=1)
                    Buffers: shared hit=100
                 -> Hash  (cost=48.84..48.84 rows=2442 width=8) (actual time=2.335..2.335 rows=2483 loops=1)
                    Buckets: 4096  Batches: 1  Memory Usage: 129kB
                    Buffers: shared hit=12 read=9
                    -> CTE Scan on recent_sales rs  (cost=0.00..48.84 rows=2442 width=8) (actual time=0.414..2.010 rows=2483 loops=1)
                       Buffers: shared hit=12 read=9
              -> Hash  (cost=13.00..13.00 rows=600 width=16) (actual time=0.194..0.194 rows=600 loops=1)
                 Buckets: 1024  Batches: 1  Memory Usage: 37kB
                 Buffers: shared hit=7
                 -> Seq Scan on products p  (cost=0.00..13.00 rows=600 width=16) (actual time=0.008..0.101 rows=600 loops=1)
                    Buffers: shared hit=7
           -> Hash  (cost=20.70..20.70 rows=1070 width=40) (actual time=0.019..0.019 rows=14 loops=1)
              Buckets: 2048  Batches: 1  Memory Usage: 17kB
              Buffers: shared hit=1
              -> Seq Scan on categories c  (cost=0.00..20.70 rows=1070 width=40) (actual time=0.008..0.010 rows=14 loops=1)
                 Buffers: shared hit=1
Planning:
 Buffers: shared hit=64
Planning Time: 2.458 ms
Execution Time: 11.534 ms
```

```sql
-- [② 실제 카테고리 Top 10 조회
WITH recent_sales AS MATERIALIZED (
    SELECT order_id
    FROM ecom.orders
    WHERE order_status IN ('paid', 'shipped', 'delivered')
      AND order_ts >= CURRENT_DATE - 90
)
SELECT c.category_id, c.category_name,
       ROUND(SUM(oi.line_total), 2) AS revenue_90d
FROM recent_sales rs
JOIN ecom.order_items oi ON oi.order_id = rs.order_id
JOIN ecom.products p ON p.product_id = oi.product_id
JOIN ecom.categories c ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY revenue_90d DESC, c.category_id
LIMIT 10;
```

categories 1

WITH recent_sales AS MATERIALIZED (SELECT order_id I

	category_id	category_name	revenue_90d
1	13	Outdoor	758,672.65
2	2	Phones	541,480.74
3	6	Appliances	138,788.46
4	14	Fitness	125,711.82
5	11	Shoes	124,829.38
6	7	Cookware	117,640.32
7	4	Audio	112,653.77
8	3	Laptops	109,500.6
9	9	Men	99,046.69
10	10	Women	96,304.36

구분	Orders 접근 방식	Join 방식	Buffers	Execution Time
튜닝 전	_____	_____	_____	_____ ms
튜닝 후	_____	_____	_____	_____ ms

Top 10 결과 및 해석: _____

6. Q4 - 제품별 누적매출 RANK Top 20

요청사항

제품별 누적매출을 먼저 계산하고 `RANK()`로 상위 20개 제품을 제공한다.

Q4 튜닝 전

ecom | *<postgres 4> Script-10 | <postgres 4> Script-11

```sql
-- ==========================================================
-- Q4. 제품별 누적매출 RANK() Top 20
-- 문제점: 600개 제품 각각에 대해 매출 상관 서브쿼리를 실행한다.
-- ==========================================================
-- [① 실행계획 확인용] 반복 SubPlan의 loops와 실행시간 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                 -- 실제 실행계획과 반복 횟수 측정
WITH product_revenue AS (                           -- 제품별 매출 중간 결과 CTE 시작
    SELECT p.product_id,
           p.product_name,
           COALESCE((                               -- 매출이 없으면 NULL 대신 0
               SELECT SUM(oi.line_total)            -- 제품별 주문상품 금액 합산
               FROM ecom.order_items oi
               JOIN ecom.orders o ON o.order_id = oi.order_id
               WHERE oi.product_id = p.product_id
                 AND o.order_status IN ('paid', 'shipped', 'delivered')
           ), 0) AS revenue
    FROM ecom.products p
), ranked AS (                                      -- 매출 결과에 순위를 붙이는 두 번째 CTE
    SELECT product_id,
           product_name,
           revenue,
           RANK() OVER (ORDER BY revenue DESC) AS revenue_rank -- 매출 내림차순 공동 순위
    FROM product_revenue
    WHERE revenue > 0
)
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank
FROM ranked
WHERE revenue_rank <= 20                            -- 순위가 20위 이내인 제품만 표시
ORDER BY revenue_rank, product_id;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH product | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Sort  (cost=158635.79..158636.29 rows=200 width=59) (actual time=36.685..36.687 rows=20 loops=1)
  Sort Key: ranked.revenue_rank, ranked.product_id
  Sort Method: quicksort  Memory: 26kB
  Buffers: shared hit=70677
  -> Subquery Scan on ranked  (cost=126903.76..158628.14 rows=200 width=59) (actual time=36.334..36.354 rows=20 loops=1)
      Buffers: shared hit=70677
      -> WindowAgg  (cost=126903.76..158627.64 rows=200 width=59) (actual time=36.332..36.349 rows=20 loops=1)
          Run Condition: (rank() OVER (?) <= 20)
          Buffers: shared hit=70677
          -> Sort  (cost=126903.74..126904.24 rows=200 width=51) (actual time=36.305..36.308 rows=21 loops=1)
              Sort Key: (COALESCE((SubPlan 1), '0'::numeric)) DESC
              Sort Method: quicksort  Memory: 52kB
              Buffers: shared hit=70677
              -> Seq Scan on products p  (cost=0.00..126896.10 rows=200 width=51) (actual time=1.089..35.299 rows=585 loops=1)
                  Filter: (COALESCE((SubPlan 2), '0'::numeric) > '0'::numeric)
                  Rows Removed by Filter: 15
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH product | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
              Buffers: shared hit=70677
              -> Seq Scan on products p  (cost=0.00..126896.10 rows=200 width=51) (actual time=1.089..35.299 rows=585 loops=1)
                  Filter: (COALESCE((SubPlan 2), '0'::numeric) > '0'::numeric)
                  Rows Removed by Filter: 15
                  Buffers: shared hit=70677
                  SubPlan 1
                    -> Aggregate  (cost=158.59..158.60 rows=1 width=32) (actual time=0.025..0.025 rows=1 loops=585)
                        Buffers: shared hit=35254
                        -> Nested Loop  (cost=4.70..158.55 rows=14 width=6) (actual time=0.006..0.024 rows=14 loops=585)
                            Buffers: shared hit=35254
                            -> Bitmap Heap Scan on order_items oi  (cost=4.42..49.43 rows=17 width=14) (actual time=0.002..0.005 row
                                Recheck Cond: (product_id = p.product_id)
                                Heap Blocks: exact=3099
                                Buffers: shared hit=4270
                                -> Bitmap Index Scan on idx_order_items_product  (cost=0.00..4.41 rows=17 width=0) (actual time=0.00
                                    Index Cond: (product_id = p.product_id)
                                    Buffers: shared hit=1171
                            -> Index Scan using orders_pkey on orders o  (cost=0.28..6.42 rows=1 width=8) (actual time=0.001..0.001 ro
                                Index Cond: (order_id = oi.order_id)
                                Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                Rows Removed by Filter: 0
                                Buffers: shared hit=30984
                  SubPlan 2
                    -> Aggregate  (cost=158.59..158.60 rows=1 width=32) (actual time=0.033..0.033 rows=1 loops=600)
                        Buffers: shared hit=35416
                        -> Nested Loop  (cost=4.70..158.55 rows=14 width=6) (actual time=0.010..0.031 rows=14 loops=600)
                            Buffers: shared hit=35416
                            -> Bitmap Heap Scan on order_items oi_1  (cost=4.42..49.43 rows=17 width=14) (actual time=0.005..0.009 ro
                                Recheck Cond: (product_id = p.product_id)
                                Heap Blocks: exact=3114
                                Buffers: shared hit=4315
                                -> Bitmap Index Scan on idx_order_items_product  (cost=0.00..4.41 rows=17 width=0) (actual time=0.00
                                    Index Cond: (product_id = p.product_id)
                                    Buffers: shared hit=1201
                            -> Index Scan using orders_pkey on orders o_1  (cost=0.28..6.42 rows=1 width=8) (actual time=0.001..0.001
                                Index Cond: (order_id = oi_1.order_id)
                                Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                Rows Removed by Filter: 0
                                Buffers: shared hit=31101
Planning:
  Buffers: shared hit=24
Planning Time: 1.451 ms
Execution Time: 37.373 ms
```

```sql
WITH product_revenue AS (                           -- [②] 실제 제품별 매출 순위 조회
    SELECT p.product_id,
           p.product_name,
           COALESCE((
               SELECT SUM(oi.line_total)
               FROM ecom.order_items oi
               JOIN ecom.orders o ON o.order_id = oi.order_id
               WHERE oi.product_id = p.product_id
                 AND o.order_status IN ('paid', 'shipped', 'delivered')
           ), 0) AS revenue
    FROM ecom.products p
), ranked AS (
    SELECT product_id, product_name, revenue,
           RANK() OVER (ORDER BY revenue DESC) AS revenue_rank
    FROM product_revenue
    WHERE revenue > 0
)
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank
FROM ranked
WHERE revenue_rank <= 20
ORDER BY revenue_rank, product_id;
```

products 1

WITH product_revenue AS (SELECT p.product_id, p.prod | Enter a SQL expression to filter results (use Ctrl

	product_id	product_name	revenue	revenue_rank
1	293	Product 465	859,785.6	1
2	455	Product 378	430,633.92	2
3	501	Product 439	9,189.04	3
4	272	Product 315	8,234.57	4
5	264	Product 225	8,109.2	5
6	528	Product 549	8,040.23	6
7	315	Product 136	7,868.31	7
8	69	Product 42	7,817.88	8
9	235	Product 114	7,584.68	9
10	533	Product 39	7,426.7	10
11	236	Product 584	7,398.53	11
12	63	Product 32	7,375.03	12
13	55	Product 481	6,944.38	13
14	137	Product 463	6,919.99	14
15	33	Product 211	6,859.89	15
16	449	Product 418	6,748.8	16
17	311	Product 96	6,699.06	17
18	181	Product 264	6,586.17	18
19	429	Product 498	6,539.14	19
20	81	Product 522	6,463.1	20

느린 이유와 개선 방법

튜닝 전에는 제품마다 주문상품과 주문을 반복 조회한다. 튜닝 후에는 전체 매출을 제품별로 한 번 집계한 후 Window Function을 적용한다.

Q4 튜닝 후

```sql
-- Q4. 전체 매출을 한 번만 집계한 뒤 RANK 적용
-- 튜닝: 제품마다 매출을 다시 찾지 않고 전체 데이터를 한 번 읽어 제품별로 묶는다.
-- [① 실행계획 확인용] 반복 SubPlan이 제거됐는지 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH product_revenue AS (                          -- 전체 데이터를 제품별로 한 번 집계
    SELECT p.product_id,
           p.product_name,
           SUM(oi.line_total) AS revenue
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    JOIN ecom.products p ON p.product_id = oi.product_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY p.product_id, p.product_name
), ranked AS (                                     -- 집계 결과에 순위를 부여
    SELECT product_id, product_name, revenue,
           RANK() OVER (ORDER BY revenue DESC) AS revenue_rank -- 매출 내림차순 공동 순위
    FROM product_revenue
)
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank
FROM ranked
WHERE revenue_rank <= 20
ORDER BY revenue_rank, product_id;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH product | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Sort  (cost=520.70..522.20 rows=600 width=59) (actual time=9.509..9.513 rows=20 loops=1)
  Sort Key: ranked.revenue_rank, ranked.product_id
  Sort Method: quicksort  Memory: 26kB
  Buffers: shared hit=147
  ->  Subquery Scan on ranked  (cost=481.03..493.01 rows=600 width=59) (actual time=9.481..9.501 rows=20 loops=1)
        Buffers: shared hit=147
        ->  WindowAgg  (cost=481.03..491.51 rows=600 width=59) (actual time=9.479..9.495 rows=20 loops=1)
              Run Condition: (rank() OVER (?) <= 20)
              Buffers: shared hit=147
              ->  Sort  (cost=481.01..482.51 rows=600 width=51) (actual time=9.473..9.477 rows=21 loops=1)
                    Sort Key: product_revenue.revenue DESC
                    Sort Method: quicksort  Memory: 52kB
                    Buffers: shared hit=147
                    ->  Subquery Scan on product_revenue  (cost=445.83..453.33 rows=600 width=51) (actual time=9.089..9.292 rows=585 loops=1)
                          Buffers: shared hit=147
                          ->  HashAggregate  (cost=445.83..453.33 rows=600 width=51) (actual time=9.088..9.241 rows=585 loops=1)
                                Group Key: p.product_id
                                Batches: 1  Memory Usage: 297kB
                                Buffers: shared hit=147
                                ->  Hash Join  (cost=151.93..404.63 rows=8240 width=25) (actual time=2.183..7.128 rows=8230 loops=1)
                                      Hash Cond: (oi.product_id = p.product_id)
                                      Buffers: shared hit=147
                                      ->  Hash Join  (cost=131.43..362.34 rows=8240 width=14) (actual time=1.843..5.248 rows=8230 loops=1)
                                            Hash Cond: (oi.order_id = o.order_id)
                                            Buffers: shared hit=140
                                            ->  Seq Scan on order_items oi  (cost=0.00..203.67 rows=10367 width=22) (actual time=0.010..1.255 rows=10367 loops=1)
                                                  Buffers: shared hit=100
                                            ->  Hash  (cost=93.08..93.08 rows=3068 width=8) (actual time=1.801..1.802 rows=3068 loops=1)
                                                  Buckets: 4096  Batches: 1  Memory Usage: 152kB
                                                  Buffers: shared hit=40
                                                  ->  Seq Scan on orders o  (cost=0.00..93.08 rows=3068 width=8) (actual time=0.012..1.281 rows=3068 loops=1)
                                                        Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                                        Rows Removed by Filter: 792
                                                        Buffers: shared hit=40
                                      ->  Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.315..0.316 rows=600 loops=1)
                                            Buckets: 1024  Batches: 1  Memory Usage: 39kB
                                            Buffers: shared hit=7
                                            ->  Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=0.014..0.147 rows=600 loops=1)
                                                  Buffers: shared hit=7
Planning:
  Buffers: shared hit=114 read=3 dirtied=1
Planning Time: 6.071 ms
Execution Time: 9.654 ms
```

```sql
WITH product_revenue AS (                          -- [②] 실제 제품별 순위 조회
    SELECT p.product_id, p.product_name, SUM(oi.line_total) AS revenue
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    JOIN ecom.products p ON p.product_id = oi.product_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY p.product_id, p.product_name
), ranked AS (
    SELECT product_id, product_name, revenue,
           RANK() OVER (ORDER BY revenue DESC) AS revenue_rank
    FROM product_revenue
)
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank
FROM ranked
WHERE revenue_rank <= 20
ORDER BY revenue_rank, product_id;
```

products 1

WITH product_revenue AS (SELECT p.product_id, p.prod | Enter a SQL expression to filter results (use C

	product_id	product_name	revenue	revenue_rank
1	293	Product 465	859,785.6	1
2	455	Product 378	430,633.92	2
3	501	Product 439	9,189.04	3
4	272	Product 315	8,234.57	4
5	264	Product 225	8,109.2	5
6	528	Product 549	8,040.23	6
7	315	Product 136	7,868.31	7
8	69	Product 42	7,817.88	8
9	235	Product 114	7,584.68	9
10	533	Product 39	7,426.7	10
11	236	Product 584	7,398.53	11
12	63	Product 32	7,375.03	12
13	55	Product 481	6,944.38	13
14	137	Product 463	6,919.99	14
15	33	Product 211	6,859.89	15
16	449	Product 418	6,748.8	16
17	311	Product 96	6,699.06	17
18	181	Product 264	6,586.17	18
19	429	Product 498	6,539.14	19
20	81	Product 522	6,463.1	20

구분	반복 SubPlan	Buffers	Execution Time	1위 제품/매출
튜닝 전	_____	_____	_____ ms	_____
튜닝 후	_____	_____	_____ ms	_____

결과 해석: _____

7. Q5 - 고객별 RFM 분석

요청사항

고객별 최근성(최근 구매일), 빈도(주문 수), 금액(총매출)을 제공한다.

Q5 튜닝 전

```sql
-- ==========================================================
-- Q5. 고객별 RFM
-- 문제점: 고객마다 최근 구매·빈도·금액을 각각 다시 조회한다.
-- 화면에는 M(금액) 상위 20명만 표시한다.
-- 의미: Recency=최근 구매 후 경과일, Frequency=구매 빈도, Monetary=구매 금액이다.
-- ==========================================================
-- [① 실행계획 확인용] 고객별 반복 서브쿼리와 loops 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                 -- 고객별 반복 서브쿼리 비용 측정
SELECT c.customer_id,
       c.full_name,
       CURRENT_DATE - (                             -- 오늘에서 최근 구매일을 빼 경과일 계산
           SELECT MAX(o.order_ts)::date             -- 고객의 가장 최근 실구매일
           FROM ecom.orders o
           WHERE o.customer_id = c.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS recency_days,                           -- R: 최근 구매 후 경과일
       (
           SELECT COUNT(*)                          -- 고객의 실구매 주문 수
           FROM ecom.orders o
           WHERE o.customer_id = c.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS frequency,                              -- F: 구매 빈도
       (
           SELECT COALESCE(SUM(oi.line_total), 0)   -- 고객의 주문상품 총매출
           FROM ecom.orders o
           JOIN ecom.order_items oi ON oi.order_id = o.order_id
           WHERE o.customer_id = c.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS monetary                                -- M: 구매 금액
FROM ecom.customers c
WHERE EXISTS (                                      -- 실구매 이력이 있는 고객만 포함
    SELECT 1
    FROM ecom.orders o
    WHERE o.customer_id = c.customer_id
      AND o.order_status IN ('paid', 'shipped', 'delivered')
)
ORDER BY monetary DESC, c.customer_id
LIMIT 20;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.cus | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	Limit (cost=50416.78..51083.45 rows=20 width=65) (actual time=22.675..22.892 rows=20 loops=1)
2	Buffers: shared hit=17287
3	-> Result (cost=50416.78..150418.03 rows=3000 width=65) (actual time=22.674..22.889 rows=20 loops=1)
4	Buffers: shared hit=17287
5	-> Sort (cost=50416.78..50424.28 rows=3000 width=53) (actual time=22.575..22.577 rows=20 loops=1)
6	Sort Key: ((SubPlan 4)) DESC, c.customer_id
7	Sort Method: top-N heapsort Memory: 26kB
8	Buffers: shared hit=16776
9	-> Hash Join (cost=159.45..50336.95 rows=3000 width=53) (actual time=3.839..22.055 rows=2217 loops=1)
10	Hash Cond: (c.customer_id = o.customer_id)
11	Buffers: shared hit=16776
12	-> Seq Scan on customers c (cost=0.00..70.00 rows=3000 width=21) (actual time=0.464..5.609 rows=3000 loops=1)
13	Buffers: shared hit=40
14	-> Hash (cost=126.84..126.84 rows=2609 width=8) (actual time=3.049..3.049 rows=2217 loops=1)
15	Buckets: 4096 Batches: 1 Memory Usage: 119kB

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.cus | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
16	Buffers: shared hit=40
17	-> HashAggregate (cost=100.75..126.84 rows=2609 width=8) (actual time=2.221..2.606 rows=2217 loops=1)
18	Group Key: o.customer_id
19	Batches: 1 Memory Usage: 241kB
20	Buffers: shared hit=40
21	-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=8) (actual time=0.020..1.367 rows=3068 loops=1)
22	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
23	Rows Removed by Filter: 792
24	Buffers: shared hit=40
25	SubPlan 4
26	-> Aggregate (cost=16.68..16.69 rows=1 width=32) (actual time=0.005..0.005 rows=1 loops=2217)
27	Buffers: shared hit=16696
28	-> Nested Loop (cost=0.57..16.67 rows=3 width=6) (actual time=0.003..0.005 rows=4 loops=2217)
29	Buffers: shared hit=16696
30	-> Index Scan using idx_orders_customer_ts on orders o_3 (cost=0.28..8.30 rows=1 width=8) (actual time=0.001..0.002 rows=1 loops=2217)
31	Index Cond: (customer_id = c.customer_id)
32	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
33	Rows Removed by Filter: 0
34	Buffers: shared hit=7438
35	-> Index Scan using idx_order_items_order on order_items oi (cost=0.29..8.34 rows=3 width=14) (actual time=0.001..0.002 rows=3 loops=3068)
36	Index Cond: (order_id = o_3.order_id)
37	Buffers: shared hit=9258
38	SubPlan 2
39	-> Result (cost=8.30..8.31 rows=1 width=4) (actual time=0.004..0.004 rows=1 loops=20)
40	Buffers: shared hit=60
41	InitPlan 1
42	-> Limit (cost=0.28..8.30 rows=1 width=8) (actual time=0.003..0.003 rows=1 loops=20)
43	Buffers: shared hit=60
44	-> Index Scan using idx_orders_customer_ts on orders o_1 (cost=0.28..8.30 rows=1 width=8) (actual time=0.003..0.003 rows=1 loops=20)
45	Index Cond: (customer_id = c.customer_id)
46	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
47	Buffers: shared hit=60
48	SubPlan 3
49	-> Aggregate (cost=8.30..8.31 rows=1 width=8) (actual time=0.010..0.010 rows=1 loops=20)
50	Buffers: shared hit=451
51	-> Index Scan using idx_orders_customer_ts on orders o_2 (cost=0.28..8.30 rows=1 width=0) (actual time=0.002..0.008 rows=23 loops=20)
52	Index Cond: (customer_id = c.customer_id)
53	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
54	Rows Removed by Filter: 0
55	Buffers: shared hit=451
56	Planning:
57	Buffers: shared hit=24
58	Planning Time: 1.897 ms
59	Execution Time: 23.222 ms

```sql
SELECT c.customer_id,                               -- [②] 실제 고객별 RFM 결과 조회
       c.full_name,
       CURRENT_DATE - (
           SELECT MAX(o.order_ts)::date FROM ecom.orders o
           WHERE o.customer_id = c.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS recency_days,
       (
           SELECT COUNT(*) FROM ecom.orders o
           WHERE o.customer_id = c.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS frequency,
       (
           SELECT COALESCE(SUM(oi.line_total), 0)
           FROM ecom.orders o
           JOIN ecom.order_items oi ON oi.order_id = o.order_id
           WHERE o.customer_id = c.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS monetary
FROM ecom.customers c
WHERE EXISTS (
    SELECT 1 FROM ecom.orders o
    WHERE o.customer_id = c.customer_id
      AND o.order_status IN ('paid', 'shipped', 'delivered')
)
ORDER BY monetary DESC, c.customer_id
LIMIT 20;
```

customers 1

SELECT c.customer_id, c.full_name, CURRENT_DATE - (S | Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	full_name	recency_days	frequency	monetary
1	8	Customer 8	2	23	35,154.22
2	16	Customer 16	1	23	31,003.24
3	18	Customer 18	3	23	30,283.04
4	14	Customer 14	6	23	29,322.59
5	6	Customer 6	0	22	28,703.23
6	13	Customer 13	2	22	28,633.41
7	2	Customer 2	8	23	28,623.31
8	1	Customer 1	3	23	28,546.26
9	20	Customer 20	0	22	28,531.13
10	9	Customer 9	1	23	28,488.91
11	17	Customer 17	8	23	27,317.53
12	24	Customer 24	0	22	27,075.74
13	23	Customer 23	4	22	25,717.57
14	30	Customer 30	0	23	24,815.39
15	4	Customer 4	1	23	24,726.24
16	12	Customer 12	4	23	22,419.91
17	19	Customer 19	0	23	22,412.27
18	22	Customer 22	1	23	22,206.5
19	7	Customer 7	1	23	22,034.39
20	29	Customer 29	0	22	21,696.95

느린 이유와 개선 방법

튜닝 전에는 고객마다 최근 구매·빈도·금액을 각각 조회한다. 튜닝 후에는 주문별 금액을 선집계하고 고객별 `MAX`, `COUNT`, `SUM`을 한 번에 계산한다.

Q5 튜닝 후

```sql
-- Q5. 주문별 매출 선집계 후 고객별 한 번의 GROUP BY로 RFM 산출
-- 튜닝: 고객 한 명마다 세 가지 서브쿼리를 반복하는 작업을 제거한다.
-- [① 실행계획 확인용] 고객별 반복 조회 대신 한 번에 집계하는지 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH order_totals AS (                          -- 주문별 금액을 먼저 집계
    SELECT o.order_id, o.customer_id, o.order_ts,
           SUM(oi.line_total) AS order_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.customer_id, o.order_ts
), rfm AS (                                     -- 주문별 결과를 고객별 RFM으로 재집계
    SELECT customer_id,
           CURRENT_DATE - MAX(order_ts)::date AS recency_days, -- 최근 구매 경과일
           COUNT(*) AS frequency,               -- 구매 빈도
           SUM(order_total) AS monetary         -- 고객 총매출
    FROM order_totals
    GROUP BY customer_id
)
SELECT c.customer_id, c.full_name,
       r.recency_days, r.frequency, ROUND(r.monetary, 2) AS monetary
FROM rfm r
JOIN ecom.customers c ON c.customer_id = r.customer_id
ORDER BY r.monetary DESC, c.customer_id
LIMIT 20;
```

```sql
WITH order_totals AS (                          -- [②] 실제 고객별 RFM 조회
    SELECT o.order_id, o.customer_id, o.order_ts,
           SUM(oi.line_total) AS order_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.customer_id, o.order_ts
), rfm AS (
    SELECT customer_id, CURRENT_DATE - MAX(order_ts)::date AS recency_days,
           COUNT(*) AS frequency, SUM(order_total) AS monetary
    FROM order_totals
    GROUP BY customer_id
)
SELECT c.customer_id, c.full_name,
       r.recency_days, r.frequency, ROUND(r.monetary, 2) AS monetary
FROM rfm r
JOIN ecom.customers c ON c.customer_id = r.customer_id
ORDER BY r.monetary DESC, c.customer_id
LIMIT 20;
```

customers 1

WITH order_totals AS (SELECT o.order_id, o.customer_id | Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	full_name	recency_days	frequency	monetary
1	8	Customer 8	2	23	35,154.22
2	16	Customer 16	1	23	31,003.24
3	18	Customer 18	3	23	30,283.04
4	14	Customer 14	6	23	29,322.59
5	6	Customer 6	0	22	28,703.23
6	13	Customer 13	2	22	28,633.41
7	2	Customer 2	8	23	28,623.31
8	1	Customer 1	3	23	28,546.26
9	20	Customer 20	0	22	28,531.13
10	9	Customer 9	1	23	28,488.91
11	17	Customer 17	8	23	27,317.53
12	24	Customer 24	0	22	27,075.74
13	23	Customer 23	4	22	25,717.57
14	30	Customer 30	0	23	24,815.39
15	4	Customer 4	1	23	24,726.24
16	12	Customer 12	4	23	22,419.91
17	19	Customer 19	0	23	22,412.27
18	22	Customer 22	1	23	22,206.5
19	7	Customer 7	1	23	22,034.39
20	29	Customer 29	0	22	21,696.95

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH order_t | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Limit  (cost=595.46..595.51 rows=20 width=97) (actual time=12.891..12.896 rows=20 loops=1)
  Buffers: shared hit=180
  -> Sort  (cost=595.46..595.96 rows=200 width=97) (actual time=12.889..12.892 rows=20 loops=1)
      Sort Key: r.monetary DESC, c.customer_id
      Sort Method: top-N heapsort  Memory: 28kB
      Buffers: shared hit=180
      -> Hash Join  (cost=511.75..590.14 rows=200 width=97) (actual time=11.383..12.458 rows=2217 loops=1)
          Hash Cond: (c.customer_id = r.customer_id)
          Buffers: shared hit=180
          -> Seq Scan on customers c  (cost=0.00..70.00 rows=3000 width=21) (actual time=0.011..0.397 rows=3000 loops=1)
              Buffers: shared hit=40
          -> Hash  (cost=509.25..509.25 rows=200 width=52) (actual time=11.339..11.341 rows=2217 loops=1)
              Buckets: 4096 (originally 1024)  Batches: 1 (originally 1)  Memory Usage: 171kB
              Buffers: shared hit=140
              -> Subquery Scan on r  (cost=503.25..509.25 rows=200 width=52) (actual time=10.035..10.950 rows=2217 loops=1)
                  Buffers: shared hit=140
                  -> HashAggregate  (cost=503.25..507.25 rows=200 width=52) (actual time=10.035..10.772 rows=2217 loops=1)
                      Group Key: o.customer_id
                      Batches: 1  Memory Usage: 1153kB
                      Buffers: shared hit=140
                      -> HashAggregate  (cost=403.54..441.89 rows=3068 width=56) (actual time=7.586..8.646 rows=3068 loops=1)
                          Group Key: o.order_id
                          Batches: 1  Memory Usage: 1393kB
                          Buffers: shared hit=140
                          -> Hash Join  (cost=131.43..362.34 rows=8240 width=30) (actual time=1.480..4.897 rows=8230 loops=1)
                              Hash Cond: (oi.order_id = o.order_id)
                              Buffers: shared hit=140
                              -> Seq Scan on order_items oi  (cost=0.00..203.67 rows=10367 width=14) (actual time=0.008..1.102 rows=
                                  Buffers: shared hit=100
                              -> Hash  (cost=93.08..93.08 rows=3068 width=24) (actual time=1.453..1.454 rows=3068 loops=1)
                                  Buckets: 4096  Batches: 1  Memory Usage: 200kB
                                  Buffers: shared hit=40
                                  -> Seq Scan on orders o  (cost=0.00..93.08 rows=3068 width=24) (actual time=0.011..0.977 rows=306
                                      Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                      Rows Removed by Filter: 792
                                      Buffers: shared hit=40
Planning:
  Buffers: shared hit=109
Planning Time: 2.882 ms
Execution Time: 13.067 ms
```

구분	집계 방식	Buffers	Execution Time	전후 결과 동일 여부
튜닝 전	고객별 상관 서브쿼리	_____	_____ ms	-
튜닝 후	주문 선집계 후 고객 GROUP BY	_____	_____ ms	_____

RFM 결과 해석: _____

8. Q6 - 첫 구매 후 30일 내 재구매율

요청사항

실구매 고객 중 첫 구매 후 30일 안에 추가 실구매가 있었던 고객의 비율을 제공한다.

Q6 튜닝 전

```sql
-- ==========================================
-- Q6. 첫 구매 후 30일 내 재구매율
-- 문제점: 고객별 첫 구매를 구한 뒤 각 고객마다 EXISTS 서브쿼리를 실행한다.
-- ==========================================
-- [① 실행계획 확인용] 고객별 EXISTS 반복 탐색 비용 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                 -- EXISTS 반복 탐색 비용 측정
WITH first_purchase AS (                            -- 고객별 첫 구매시각 CTE
    SELECT customer_id, MIN(order_ts) AS first_ts   -- MIN으로 최초 구매시각 계산
    FROM ecom.orders
    WHERE order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY customer_id
)
SELECT COUNT(*) AS purchasing_customers,
       COUNT(*) FILTER (WHERE EXISTS (              -- 30일 내 추가 주문이 있는 고객만 계산
           SELECT 1
           FROM ecom.orders o2
           WHERE o2.customer_id = fp.customer_id
             AND o2.order_status IN ('paid', 'shipped', 'delivered')
             AND o2.order_ts > fp.first_ts
             AND o2.order_ts <= fp.first_ts + INTERVAL '30 days' -- 첫 구매 후 30일까지
       )) AS repurchased_customers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1
           FROM ecom.orders o3
           WHERE o3.customer_id = fp.customer_id
             AND o3.order_status IN ('paid', 'shipped', 'delivered')
             AND o3.order_ts > fp.first_ts
             AND o3.order_ts <= fp.first_ts + INTERVAL '30 days'
       )) / NULLIF(COUNT(*), 0), 2) AS repurchase_rate_pct -- 0 나누기 방지 후 재구매율 계산
FROM first_purchase fp;
```

```sql
WITH first_purchase AS (                            -- [②] 실제 재구매율 결과 조회
    SELECT customer_id, MIN(order_ts) AS first_ts
    FROM ecom.orders
    WHERE order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY customer_id
)
SELECT COUNT(*) AS purchasing_customers,
       COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM ecom.orders o2
           WHERE o2.customer_id = fp.customer_id
             AND o2.order_status IN ('paid', 'shipped', 'delivered')
             AND o2.order_ts > fp.first_ts
             AND o2.order_ts <= fp.first_ts + INTERVAL '30 days'
       )) AS repurchased_customers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE EXISTS (
           SELECT 1 FROM ecom.orders o3
           WHERE o3.customer_id = fp.customer_id
             AND o3.order_status IN ('paid', 'shipped', 'delivered')
             AND o3.order_ts > fp.first_ts
             AND o3.order_ts <= fp.first_ts + INTERVAL '30 days'
       )) / NULLIF(COUNT(*), 0), 2) AS repurchase_rate_pct
FROM first_purchase fp;
```

first_purchase 1

WITH first_purchase AS (SELECT customer_id, MIN(order | Enter a SQL expression to filter results (use Ctrl+Space)

	purchasing_customers	repurchased_customers	repurchase_rate_pct
1	2,217	226	10.19

```
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH first_pu
QUERY PLAN
Aggregate  (cost=43509.13..43509.16 rows=1 width=48) (actual time=13.274..13.275 rows=1 loops=1)
  Buffers: shared hit=9362
  -> HashAggregate  (cost=108.42..134.50 rows=2609 width=16) (actual time=3.044..3.566 rows=2217 loops=1)
      Group Key: orders.customer_id
      Batches: 1  Memory Usage: 369kB
      Buffers: shared hit=40
      -> Seq Scan on orders  (cost=0.00..93.08 rows=3068 width=16) (actual time=0.076..1.737 rows=3068 loops=1)
          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
          Rows Removed by Filter: 792
          Buffers: shared hit=40
  SubPlan 1
    -> Index Scan using idx_orders_customer_ts on orders o2  (cost=0.28..8.31 rows=1 width=0) (actual time=0.002..0.002 rows=0 loops=2217)
        Index Cond: ((customer_id = orders.customer_id) AND (order_ts > (min(orders.order_ts))) AND (order_ts <= ((min(orders.order_ts)) + '30 days'::interval)))
        Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
        Rows Removed by Filter: 0
        Buffers: shared hit=4661
  SubPlan 2
    -> Index Scan using idx_orders_customer_ts on orders o3  (cost=0.28..8.31 rows=1 width=0) (actual time=0.001..0.001 rows=0 loops=2217)
        Index Cond: ((customer_id = orders.customer_id) AND (order_ts > (min(orders.order_ts))) AND (order_ts <= ((min(orders.order_ts)) + '30 days'::interval)))
        Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
        Rows Removed by Filter: 0
        Buffers: shared hit=4661
Planning Time: 0.171 ms
Execution Time: 13.510 ms
```

느린 이유와 개선 방법

튜닝 전에는 고객별 첫 구매 이후 주문을 `EXISTS`로 반복 탐색한다. 튜닝 후에는 Window Function으로 첫 구매시각을 표시하고 고객별 `BOOL_OR`로 재구매 여부를 한 번 집계한다.

Q6 튜닝 후

```
-- Q6. 윈도 함수로 첫 구매시각을 한 번 표시한 뒤 고객별 BOOL_OR 집계
  -- [① 실행계획 확인용] Window와 BOOL_OR 집계 계획 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH purchase_sequence AS (
    SELECT customer_id,
           order_ts,
           MIN(order_ts) OVER (PARTITION BY customer_id) AS first_ts -- 각 주문행에 고객 첫 구매시각 표시
    FROM ecom.orders
    WHERE order_status IN ('paid', 'shipped', 'delivered')
), customer_flags AS (
    SELECT customer_id,
           BOOL_OR(order_ts > first_ts                  -- 조건을 만족하는 추가 주문이 하나라도 있으면 TRUE
                   AND order_ts <= first_ts + INTERVAL '30 days') AS repurchased_30d
    FROM purchase_sequence
    GROUP BY customer_id
)
SELECT COUNT(*) AS purchasing_customers,
       COUNT(*) FILTER (WHERE repurchased_30d) AS repurchased_customers,
       ROUND(100.0 * COUNT(*) FILTER (WHERE repurchased_30d)
             / NULLIF(COUNT(*), 0), 2) AS repurchase_rate_pct
FROM customer_flags;
```

```
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH purchas
QUERY PLAN
Aggregate  (cost=274.27..274.29 rows=1 width=48) (actual time=4.271..4.272 rows=1 loops=1)
  Buffers: shared hit=1 read=20
  -> GroupAggregate  (cost=0.28..238.51 rows=2384 width=9) (actual time=1.022..4.130 rows=2217 loops=1)
      Group Key: orders.customer_id
      Buffers: shared hit=1 read=20
      -> WindowAgg  (cost=0.28..176.32 rows=3068 width=24) (actual time=0.990..3.344 rows=3068 loops=1)
          Buffers: shared hit=1 read=20
          -> Index Only Scan using idx_orders_sales_customer_ts_order on orders  (cost=0.28..130.30 rows=3068 width=16) (actual time=0.971..2.039 rows=3068 loops=1)
              Heap Fetches: 0
              Buffers: shared hit=1 read=20
Planning:
  Buffers: shared hit=3
Planning Time: 0.202 ms
Execution Time: 4.316 ms
```

```
WITH purchase_sequence AS ( SELECT customer_id, order_ts, MIN(ordei
```

purchasing_customers	repurchased_customers	repurchase_rate_pct
2,217	226	10.19

구분	구매고객	30일 내 재구매고객	재구매율	Execution Time
튜닝 전	_____	_____	_____%	_____ ms
튜닝 후	_____	_____	_____%	_____ ms

결과 해석: _____

9. Q7 - 재주문 임계치 미만 상품

요청사항

`qty_on_hand < reorder_point`인 상품과 부족수량을 제공한다.

Q7 튜닝 전

```
-- ==========================================================
-- Q7. 재고가 재주문 임계치보다 낮은 상품
-- 문제점: 각 행마다 같은 inventory 행의 임계치를 상관 조회한다.
-- ==========================================================
-- [① 실행계획 확인용] inventory 반복 조회와 Scan 방식 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                 -- 재고 테이블 접근 방식과 비용 측정
SELECT p.product_id, p.sku, p.product_name,
       i.qty_on_hand, i.reorder_point,              -- 현재고와 재주문 기준 수량
       i.reorder_point - i.qty_on_hand AS shortage_qty -- 보충이 필요한 부족수량
FROM ecom.products p
JOIN ecom.inventory i ON i.product_id = p.product_id
WHERE i.qty_on_hand < (                             -- 현재고가 재주문 기준보다 낮은 행
    SELECT MAX(i2.reorder_point)                    -- 같은 재고행의 기준을 다시 조회
    FROM ecom.inventory i2
    WHERE i2.product_id = i.product_id
)
ORDER BY shortage_qty DESC, p.product_id
LIMIT 20;
```

```
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT p.proi
QUERY PLAN
Limit  (cost=5022.35..5022.40 rows=20 width=42) (actual time=1.923..1.927 rows=20 loops=1)
  Buffers: shared hit=1912 dirtied=4
  -> Sort  (cost=5022.35..5022.85 rows=200 width=42) (actual time=1.922..1.925 rows=20 loops=1)
      Sort Key: ((i.reorder_point - i.qty_on_hand)) DESC, p.product_id
      Sort Method: top-N heapsort  Memory: 27kB
      Buffers: shared hit=1912 dirtied=4
      -> Hash Join  (cost=20.50..5017.02 rows=200 width=42) (actual time=0.684..1.874 rows=66 loops=1)
          Hash Cond: (i.product_id = p.product_id)
          Buffers: shared hit=1912 dirtied=4
          -> Seq Scan on inventory i  (cost=0.00..4995.50 rows=200 width=16) (actual time=0.058..1.227 rows=66 loops=1)
              Filter: (qty_on_hand < (SubPlan 1))
              Rows Removed by Filter: 534
              Buffers: shared hit=1905 dirtied=4
              SubPlan 1
                -> Aggregate  (cost=8.29..8.30 rows=1 width=4) (actual time=0.002..0.002 rows=1 loops=600)
                    Buffers: shared hit=1900 dirtied=2
                    -> Index Scan using inventory_pkey on inventory i2  (cost=0.28..8.29 rows=1 width=4) (actual time=0.001..0.001 rows=1 loops=600)
                        Index Cond: (product_id = i.product_id)
                        Buffers: shared hit=1900 dirtied=2
          -> Hash  (cost=13.00..13.00 rows=600 width=30) (actual time=0.598..0.599 rows=600 loops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 45kB
              Buffers: shared hit=7
              -> Seq Scan on products p  (cost=0.00..13.00 rows=600 width=30) (actual time=0.005..0.456 rows=600 loops=1)
                  Buffers: shared hit=7
Planning:
  Buffers: shared hit=42 dirtied=4
Planning Time: 4.890 ms
Execution Time: 2.017 ms
```

```
SELECT p.product_id, p.sku, p.product_name,          -- [②] 실제 부족 재고 상품 조회
       i.qty_on_hand, i.reorder_point,
       i.reorder_point - i.qty_on_hand AS shortage_qty
FROM ecom.products p
JOIN ecom.inventory i ON i.product_id = p.product_id
WHERE i.qty_on_hand < (
    SELECT MAX(i2.reorder_point) FROM ecom.inventory i2
    WHERE i2.product_id = i.product_id
)
ORDER BY shortage_qty DESC, p.product_id
LIMIT 20;
-- ==========================================================
```

```
SELECT p.product_id, p.sku, p.product_name, i.qty_on_ha
```

product_id	sku	product_name	qty_on_hand	reorder_point	shortage_qty
320	SKU-000186	Product 186	0	30	30
454	SKU-000428	Product 428	0	30	30
102	SKU-000022	Product 22	1	30	29
136	SKU-000163	Product 163	1	30	29
188	SKU-000594	Product 594	1	30	29
277	SKU-000015	Product 15	1	30	29
596	SKU-000590	Product 590	1	30	29
226	SKU-000374	Product 374	2	30	28
387	SKU-000447	Product 447	2	30	28
520	SKU-000519	Product 519	2	30	28
540	SKU-000359	Product 359	2	30	28
276	SKU-000135	Product 135	3	30	27
335	SKU-000336	Product 336	3	30	27
25	SKU-000231	Product 231	4	30	26
203	SKU-000404	Product 404	4	30	26
456	SKU-000578	Product 578	4	30	26
485	SKU-000279	Product 279	4	30	26
498	SKU-000429	Product 429	4	30	26
530	SKU-000559	Product 559	4	30	26
159	SKU-000373	Product 373	5	30	25

느린 이유와 개선 방법

튜닝 전에는 같은 재고 행의 임계치를 다시 상관 조회한다. 튜닝 후에는 직접 비교 조건과 저재고 행만 포함하는 부분 인덱스를 사용한다.

Q7 튜닝 후

```
-- Q7. 직접 조건 + 저재고 부분 인덱스
-- 튜닝: 저재고 행만 저장한 부분 인덱스를 이용해 필요한 후보를 빠르게 찾는다.
-- [① 실행계획 확인용] 저재고 부분 인덱스 사용 여부 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
SELECT p.product_id, p.sku, p.product_name,
       i.qty_on_hand, i.reorder_point,
       i.reorder_point - i.qty_on_hand AS shortage_qty
FROM ecom.inventory i
JOIN ecom.products p ON p.product_id = i.product_id
WHERE i.qty_on_hand < i.reorder_point          -- 현재고가 재주문 기준보다 낮은 행만 필터
ORDER BY shortage_qty DESC, p.product_id
LIMIT 20;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT p.product_id, p.sku, p

	QUERY PLAN
1	Limit (cost=35.41..35.46 rows=20 width=42) (actual time=0.392..0.399 rows=20 loops=1)
2	Buffers: shared hit=12
3	-> Sort (cost=35.41..35.91 rows=200 width=42) (actual time=0.391..0.395 rows=20 loops=1)
4	Sort Key: ((i.reorder_point - i.qty_on_hand)) DESC, p.product_id
5	Sort Method: top-N heapsort Memory: 27kB
6	Buffers: shared hit=12
7	-> Hash Join (cost=15.00..30.09 rows=200 width=42) (actual time=0.154..0.351 rows=66 loops=1)
8	Hash Cond: (p.product_id = i.product_id)
9	Buffers: shared hit=12
10	-> Seq Scan on products p (cost=0.00..13.00 rows=600 width=30) (actual time=0.010..0.094 rows=600 loops=1)
11	Buffers: shared hit=7
12	-> Hash (cost=12.50..12.50 rows=200 width=16) (actual time=0.132..0.132 rows=66 loops=1)
13	Buckets: 1024 Batches: 1 Memory Usage: 12kB
14	Buffers: shared hit=5
15	-> Seq Scan on inventory i (cost=0.00..12.50 rows=200 width=16) (actual time=0.011..0.112 rows=66 loops=1)
16	Filter: (qty_on_hand < reorder_point)
17	Rows Removed by Filter: 534
18	Buffers: shared hit=5
19	Planning:
20	Buffers: shared hit=12
21	Planning Time: 0.379 ms
22	Execution Time: 0.428 ms

```
SELECT p.product_id, p.sku, p.product_name,        -- [②] 실제 부족 재고 상품 조회
       i.qty_on_hand, i.reorder_point,
       i.reorder_point - i.qty_on_hand AS shortage_qty
FROM ecom.inventory i
JOIN ecom.products p ON p.product_id = i.product_id
WHERE i.qty_on_hand < i.reorder_point
ORDER BY shortage_qty DESC, p.product_id
LIMIT 20;
```

products(+) 1

SELECT p.product_id, p.sku, p.product_name, i.qty_on_hand, i.reorder_pc

	product_id	sku	product_name	qty_on_hand	reorder_point	shortage_qty
1	320	SKU-000186	Product 186	0	30	30
2	454	SKU-000428	Product 428	0	30	30
3	102	SKU-000022	Product 22	1	30	29
4	136	SKU-000163	Product 163	1	30	29
5	188	SKU-000594	Product 594	1	30	29
6	277	SKU-000015	Product 15	1	30	29
7	596	SKU-000590	Product 590	1	30	29
8	226	SKU-000374	Product 374	2	30	28
9	387	SKU-000447	Product 447	2	30	28
10	520	SKU-000519	Product 519	2	30	28
11	540	SKU-000359	Product 359	2	30	28
12	276	SKU-000135	Product 135	3	30	27
13	335	SKU-000336	Product 336	3	30	27
14	25	SKU-000231	Product 231	4	30	26
15	203	SKU-000404	Product 404	4	30	26
16	456	SKU-000578	Product 578	4	30	26
17	485	SKU-000279	Product 279	4	30	26
18	498	SKU-000429	Product 429	4	30	26
19	530	SKU-000559	Product 559	4	30	26
20	159	SKU-000373	Product 373	5	30	25

구분	Inventory 접근 방식	반복 횟수	Buffers	Execution Time
튜닝 전	_____	_____	_____	_____ ms
튜닝 후	_____	_____	_____	_____ ms

부족 상품 결과 해석: _____

10. Q8 - 리뷰 기반 효자상품

요청사항

평균 평점 4.5 이상이고 리뷰가 50개 이상인 제품을 제공한다.

Q8 튜닝 전

```
-- ============================================================
-- Q8. 평점 4.5 이상, 리뷰 50개 이상 효자상품
-- 문제점: 상품마다 AVG와 COUNT를 SELECT와 WHERE에서 반복 계산한다.
-- ============================================================
-- [① 실행계획 확인용] 제품별 리뷰 반복 집계 횟수 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                -- 반복되는 리뷰 서브쿼리 비용 측정
SELECT p.product_id,
       p.product_name,
       ROUND((SELECT AVG(r.rating) FROM ecom.reviews r
              WHERE r.product_id = p.product_id), 2) AS avg_rating, -- 제품별 평균평점
       (SELECT COUNT(*) FROM ecom.reviews r
        WHERE r.product_id = p.product_id) AS review_count -- 제품별 리뷰 수
FROM ecom.products p
WHERE (SELECT AVG(r.rating) FROM ecom.reviews r      -- 평균평점을 다시 계산
       WHERE r.product_id = p.product_id) >= 4.5
  AND (SELECT COUNT(*) FROM ecom.reviews r           -- 리뷰 수를 다시 계산
       WHERE r.product_id = p.product_id) >= 50
ORDER BY avg_rating DESC, review_count DESC, p.product_id;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT p.prc

	QUERY PLAN
1	Sort (cost=20060.25..20060.42 rows=67 width=59) (actual time=2.084..2.088 rows=12 loops=1)
2	Sort Key: (round((SubPlan 1), 2)) DESC, ((SubPlan 2)) DESC, p.product_id
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=1385
5	-> Seq Scan on products p (cost=0.00..20058.22 rows=67 width=59) (actual time=0.287..2.071 rows=12 loops=1)
6	Filter: (((SubPlan 4) >= 50) AND ((SubPlan 3) >= 4.5))
7	Rows Removed by Filter: 588
8	Buffers: shared hit=1385
9	SubPlan 1
10	-> Aggregate (cost=25.43..25.44 rows=1 width=32) (actual time=0.028..0.028 rows=1 loops=12)
11	Buffers: shared hit=66
12	-> Bitmap Heap Scan on reviews r (cost=4.40..25.39 rows=16 width=4) (actual time=0.008..0.017 rows=160 loops=12)
13	Recheck Cond: (product_id = p.product_id)
14	Heap Blocks: exact=32
15	Buffers: shared hit=66
16	-> Bitmap Index Scan on reviews_product_id_customer_id_key (cost=0.00..4.40 rows=16 width=0) (actual time=0.007..0.007 rows=160 loops=12)
17	Index Cond: (product_id = p.product_id)
18	Buffers: shared hit=34
19	SubPlan 2
20	-> Aggregate (cost=4.60..4.61 rows=1 width=8) (actual time=0.021..0.021 rows=1 loops=12)
21	Buffers: shared hit=35
22	-> Index Only Scan using reviews_product_id_customer_id_key on reviews r_1 (cost=0.28..4.56 rows=16 width=0) (actual time=0.004..0.014 rows=160 loops=12)
23	Index Cond: (product_id = p.product_id)
24	Heap Fetches: 0
25	Buffers: shared hit=35
26	SubPlan 4
27	-> Aggregate (cost=4.60..4.61 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=600)
28	Buffers: shared hit=1211
29	-> Index Only Scan using reviews_product_id_customer_id_key on reviews r_3 (cost=0.28..4.56 rows=16 width=0) (actual time=0.001..0.001 rows=3 loops=600)
30	Index Cond: (product_id = p.product_id)
31	Heap Fetches: 0
32	Buffers: shared hit=1211

```
SELECT p.product_id,                                  -- [②] 실제 효자상품 결과 조회
       p.product_name,
       ROUND((SELECT AVG(r.rating) FROM ecom.reviews r
              WHERE r.product_id = p.product_id), 2) AS avg_rating,
       (SELECT COUNT(*) FROM ecom.reviews r
        WHERE r.product_id = p.product_id) AS review_count
FROM ecom.products p
WHERE (SELECT AVG(r.rating) FROM ecom.reviews r
       WHERE r.product_id = p.product_id) >= 4.5
  AND (SELECT COUNT(*) FROM ecom.reviews r
       WHERE r.product_id = p.product_id) >= 50
ORDER BY avg_rating DESC, review_count DESC, p.product_id;
```

products 1

SELECT p.product_id, p.product_name, ROUND((SELECT

	product_id	product_name	avg_rating	review_count
1	57	Product 491	4.85	160
2	229	Product 464	4.83	160
3	163	Product 253	4.81	160
4	263	Product 285	4.79	160
5	426	Product 258	4.79	160
6	243	Product 395	4.78	160
7	268	Product 25	4.77	160
8	297	Product 75	4.76	160
9	481	Product 189	4.76	160
10	356	Product 556	4.75	161
11	178	Product 113	4.74	160
12	114	Product 262	4.69	160

```
33      SubPlan 3
34        -> Aggregate  (cost=25.43..25.44 rows=1 width=32) (actual time=0.031..0.031 rows=1 loops=12)
35            Buffers: shared hit=66
36            -> Bitmap Heap Scan on reviews r_2  (cost=4.40..25.39 rows=16 width=4) (actual time=0.010..0.021 rows=160 loops=12)
37                Recheck Cond: (product_id = p.product_id)
38                Heap Blocks: exact=32
39                Buffers: shared hit=66
40                -> Bitmap Index Scan on reviews_product_id_customer_id_key  (cost=0.00..4.40 rows=16 width=0) (actual time=0.006..0.006 rows=160 loops=12)
41                    Index Cond: (product_id = p.product_id)
42                    Buffers: shared hit=34
43  Planning Time: 0.268 ms
44  Execution Time: 2.229 ms
```

느린 이유와 개선 방법

튜닝 전에는 제품별 평균과 건수를 SELECT와 WHERE에서 반복 계산한다. 튜닝 후에는 리뷰를 한 번 읽고 `GROUP BY`와 `HAVING`으로 조건을 적용한다.

Q8 튜닝 후

```
-- Q8. 리뷰 테이블을 한 번만 읽고 GROUP BY/HAVING 적용
-- 튜닝: 제품별 AVG와 COUNT를 한 번 계산하고 HAVING에서 두 조건을 함께 적용한다.
-- [① 실행계획 확인용] reviews를 한 번만 집계하는지 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
SELECT p.product_id,
       p.product_name,
       ROUND(AVG(r.rating), 2) AS avg_rating,        -- 제품별 평균평점
       COUNT(*) AS review_count                      -- 제품별 리뷰 수
FROM ecom.reviews r
JOIN ecom.products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name
HAVING AVG(r.rating) >= 4.5                          -- 집계 후 평균평점 조건 적용
   AND COUNT(*) >= 50                                -- 집계 후 리뷰 수 조건 적용
ORDER BY avg_rating DESC, review_count DESC, p.product_id;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT p.product_id, p.produ | Enter a SQL expression to filter results (use Ctrl+Space)

```
1  Sort  (cost=95.27..95.44 rows=67 width=59) (actual time=1.308..1.310 rows=12 loops=1)
2    Sort Key: (round(avg(r.rating), 2)) DESC, (count(*)) DESC, p.product_id
3    Sort Method: quicksort  Memory: 25kB
4    Buffers: shared hit=28
5    -> HashAggregate  (cost=82.57..93.24 rows=67 width=59) (actual time=1.244..1.284 rows=12 loops=1)
6        Group Key: p.product_id
7        Filter: ((avg(r.rating) >= 4.5) AND (count(*) >= 50))
8        Batches: 1  Memory Usage: 73kB
9        Rows Removed by Filter: 118
10       Buffers: shared hit=28
11       -> Hash Join  (cost=20.50..67.28 rows=2039 width=23) (actual time=0.236..0.884 rows=2039 loops=1)
12           Hash Cond: (r.product_id = p.product_id)
13           Buffers: shared hit=28
14           -> Seq Scan on reviews r  (cost=0.00..41.39 rows=2039 width=12) (actual time=0.010..0.239 rows=2039 loops=1)
15               Buffers: shared hit=21
16           -> Hash  (cost=13.00..13.00 rows=600 width=19) (actual time=0.208..0.208 rows=600 loops=1)
17               Buckets: 1024  Batches: 1  Memory Usage: 39kB
18               Buffers: shared hit=7
19               -> Seq Scan on products p  (cost=0.00..13.00 rows=600 width=19) (actual time=0.009..0.100 rows=600 loops=1)
20                   Buffers: shared hit=7
21 Planning:
22   Buffers: shared hit=15
23 Planning Time: 0.504 ms
24 Execution Time: 1.389 ms
```

```
SELECT p.product_id, p.product_name,                 -- [②] 실제 효자상품 결과 조회
       ROUND(AVG(r.rating), 2) AS avg_rating,
       COUNT(*) AS review_count
FROM ecom.reviews r
JOIN ecom.products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name
HAVING AVG(r.rating) >= 4.5
   AND COUNT(*) >= 50
ORDER BY avg_rating DESC, review_count DESC, p.product_id;
```

products 1

SELECT p.product_id, p.product_name, ROUND(AVG(r.rating), 2) AS avg_ | Enter a SQL expression to filter re

	product_id	product_name	avg_rating	review_count
1	57	Product 491	4.85	160
2	229	Product 464	4.83	160
3	163	Product 253	4.81	160
4	263	Product 285	4.79	160
5	426	Product 258	4.79	160
6	243	Product 395	4.78	160
7	268	Product 25	4.77	160
8	297	Product 75	4.76	160
9	481	Product 189	4.76	160
10	356	Product 556	4.75	161
11	178	Product 113	4.74	160
12	114	Product 262	4.69	160

구분	Reviews 접근 방식	Buffers	Execution Time	조건 충족 상품 수
튜닝 전	_____	_____	_____ ms	_____
튜닝 후	_____	_____	_____ ms	_____

결과 해석: _____

11. Q9 - 쿠폰 사용 여부별 AOV

요청사항

쿠폰 사용 주문과 미사용 주문의 주문당 평균 금액을 비교한다.

Q9 튜닝 전

```
-- ==========================================================
-- Q9. 쿠폰 사용 여부별 평균 주문금액
-- 문제점: 상세 행 조인 후 COUNT(DISTINCT)로 주문 수를 복원한다.
-- 의미: AOV는 전체 매출을 중복 없는 주문 수로 나눈 주문당 평균금액이다.
-- ==========================================================
-- [① 실행계획 확인용] 상세행 조인과 DISTINCT 집계 비용 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                  -- 상세행 조인과 중복 제거 비용 측정
SELECT CASE WHEN o.coupon_code IS NULL THEN '미사용' ELSE '사용' END AS coupon_group, -- 쿠폰 여부 분류
       COUNT(DISTINCT o.order_id) AS order_count,    -- 중복을 제거한 주문 수
       ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov -- 주문당 평균금액
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY 1
ORDER BY 1;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT CASE | Enter a SQL expression to filter results (use Ctrl+Space)

```
1  GroupAggregate  (cost=898.29..980.71 rows=1 width=72) (actual time=21.599..21.982 rows=2 loops=1)
2    Group Key: (CASE WHEN (o.coupon_code IS NULL) THEN '미사용'::text ELSE '사용'::text END)
3    Buffers: shared hit=140
4    -> Sort  (cost=898.29..918.89 rows=8240 width=46) (actual time=20.671..20.933 rows=8230 loops=1)
5        Sort Key: (CASE WHEN (o.coupon_code IS NULL) THEN '미사용'::text ELSE '사용'::text END), o.order_id
6        Sort Method: quicksort  Memory: 750kB
7        Buffers: shared hit=140
8        -> Hash Join  (cost=131.43..362.34 rows=8240 width=46) (actual time=3.026..7.213 rows=8230 loops=1)
9            Hash Cond: (oi.order_id = o.order_id)
10           Buffers: shared hit=140
11           -> Seq Scan on order_items oi  (cost=0.00..203.67 rows=10367 width=14) (actual time=0.011..1.338 rows=10367 loops=1)
12               Buffers: shared hit=100
13           -> Hash  (cost=93.08..93.08 rows=3068 width=15) (actual time=2.967..2.967 rows=3068 loops=1)
14               Buckets: 4096  Batches: 1  Memory Usage: 158kB
15               Buffers: shared hit=40
16               -> Seq Scan on orders o  (cost=0.00..93.08 rows=3068 width=15) (actual time=0.014..1.376 rows=3068 loops=1)
17                   Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
18                   Rows Removed by Filter: 792
19                   Buffers: shared hit=40
20 Planning:
21   Buffers: shared hit=15
22 Planning Time: 0.875 ms
23 Execution Time: 22.034 ms
```

```
SELECT CASE WHEN o.coupon_code IS NULL THEN '미사용' ELSE '사용' END AS coupon_group, -- [②] 실제 비교 결과
       COUNT(DISTINCT o.order_id) AS order_count,
       ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY 1
ORDER BY 1;
```

orders(+) 1

SELECT CASE WHEN o.coupon_code IS NULL THEN '미사용 | Enter a SQL expression to filter results (use Ctrl+Space)

	coupon_group	order_count	aov
1	미사용	2,307	619.55
2	사용	761	1,694.79

느린 이유와 개선 방법

상세행 조인 후에는 주문 수 복원을 위해 `COUNT(DISTINCT)`가 필요하다. 튜닝 후에는 주문별 금액을 먼저 계산한 뒤 쿠폰 사용 여부로 그룹핑해 `AVG(order_total)`을 계산한다.

Q9 튜닝 후

```
-- Q9. 주문별 매출 선집계로 DISTINCT 제거
-- 튜닝: 주문별 금액을 먼저 한 행으로 만든 뒤 그룹별 AVG(order_total)을 계산한다.
-- [① 실행계획 확인용] 주문별 선집계 후 AVG 계산 계획 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH order_totals AS (
    SELECT o.order_id,
           (o.coupon_code IS NOT NULL) AS used_coupon, -- 쿠폰 사용 여부를 TRUE/FALSE로 변환
           SUM(oi.line_total) AS order_total        -- 주문별 총금액
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.coupon_code
)
SELECT CASE WHEN used_coupon THEN '사용' ELSE '미사용' END AS coupon_group,
       COUNT(*) AS order_count,
       ROUND(AVG(order_total), 2) AS aov            -- 주문별 금액의 평균
FROM order_totals
GROUP BY used_coupon
ORDER BY coupon_group;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH order_totals AS (SELEC | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	Sort (cost=495.62..495.63 rows=2 width=73) (actual time=13.147..13.149 rows=2 loops=1)
2	Sort Key: (CASE WHEN ((o.coupon_code IS NOT NULL)) THEN '사용'::text ELSE '미사용'::text END)
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=140
5	-> HashAggregate (cost=495.58..495.61 rows=2 width=73) (actual time=13.071..13.074 rows=2 loops=1)
6	Group Key: (o.coupon_code IS NOT NULL)
7	Batches: 1 Memory Usage: 24kB
8	Buffers: shared hit=140
9	-> HashAggregate (cost=403.54..441.89 rows=3068 width=48) (actual time=11.327..12.401 rows=3068 loops=1)
10	Group Key: o.order_id
11	Batches: 1 Memory Usage: 1393kB
12	Buffers: shared hit=140
13	-> Hash Join (cost=131.43..362.34 rows=8240 width=21) (actual time=2.200..7.476 rows=8230 loops=1)
14	Hash Cond: (oi.order_id = o.order_id)
15	Buffers: shared hit=140
16	-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.010..1.664 rows=10367 loops=1)
17	Buffers: shared hit=100
18	-> Hash (cost=93.08..93.08 rows=3068 width=15) (actual time=2.169..2.170 rows=3068 loops=1)
19	Buckets: 4096 Batches: 1 Memory Usage: 158kB
20	Buffers: shared hit=40
21	-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=15) (actual time=0.007..1.535 rows=3068 loops=1)
22	Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
23	Rows Removed by Filter: 792
24	Buffers: shared hit=40
25	Planning:
26	Buffers: shared hit=22
27	Planning Time: 0.453 ms
28	Execution Time: 13.294 ms

```
WITH order_totals AS (                          -- [②] 실제 쿠폰 그룹 비교 결과 조회
    SELECT o.order_id, (o.coupon_code IS NOT NULL) AS used_coupon,
           SUM(oi.line_total) AS order_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.coupon_code
)
SELECT CASE WHEN used_coupon THEN '사용' ELSE '미사용' END AS coupon_group,
       COUNT(*) AS order_count, ROUND(AVG(order_total), 2) AS aov
FROM order_totals
GROUP BY used_coupon
ORDER BY coupon_group;
```

Results 1

WITH order_totals AS (SELECT o.order_id, (o.coupon_code IS NOT NULL | Enter a SQL expression to filter results (use C

	coupon_group	order_count	aov
1	미사용	2,307	619.55
2	사용	761	1,694.79

그룹	주문 수	AOV	해석
쿠폰 미사용	_____	_____	_____
쿠폰 사용	_____	_____	_____

구분	Buffers	Execution Time
튜닝 전	_____	_____ ms
튜닝 후	_____	_____ ms

> 쿠폰 사용과 높은 AOV의 상관관계만 확인할 수 있으며, 쿠폰의 인과효과를 의미하지는 않는다.

12. Q10 - 누적매출 상위 1% 고객의 최근 60일 매출

요청사항

전체 고객 3,000명을 누적매출로 정렬해 상위 1%인 정확히 30명을 먼저 선정하고, 선정된 고객의 최근 60일 매출을 제공한다.

Q10 튜닝 전

```
-- ==========================================================
-- Q10. 누적매출 상위 1% 고객의 최근 60일 매출
-- 문제점: 3,000명 각각의 누적매출을 상관 서브쿼리로 계산하고,
--         선정된 상위 30명의 최근 60일 매출도 다시 상관 조회한다.
-- 기준: 전체 고객 3,000명의 누적매출로 정확히 30명을 먼저 선정한다.
-- 주의: 누적매출로 30명을 먼저 선정한 다음, 그 고객들의 최근 60일 매출을 표시한다.
-- ==========================================================
-- [① 실행계획 확인용] 고객별 반복 집계와 최근 60일 SubPlan 비용 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)                 -- 고객별 반복 집계 비용 측정
WITH lifetime_sales AS (                            -- 고객별 누적매출 CTE
    SELECT c.customer_id,
           c.full_name,
           (
               SELECT COALESCE(SUM(oi.line_total), 0) -- 고객 한 명의 누적매출 계산
               FROM ecom.orders o
               JOIN ecom.order_items oi ON oi.order_id = o.order_id
               WHERE o.customer_id = c.customer_id
                 AND o.order_status IN ('paid', 'shipped', 'delivered')
           ) AS lifetime_revenue
    FROM ecom.customers c
), ranked AS (                                      -- 누적매출 순위 계산 CTE
    SELECT customer_id,
           full_name,
           lifetime_revenue,
           ROW_NUMBER() OVER (                      -- 중복 없는 순번을 1부터 부여
               ORDER BY lifetime_revenue DESC, customer_id
           ) AS lifetime_rank,
           COUNT(*) OVER () AS customer_count       -- 전체 고객 수를 각 행에 표시
    FROM lifetime_sales
)
SELECT r.customer_id,
       r.full_name,
       r.lifetime_rank,
       ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,
       ROUND((
           SELECT COALESCE(SUM(oi.line_total), 0)
           FROM ecom.orders o
           JOIN ecom.order_items oi ON oi.order_id = o.order_id
           WHERE o.customer_id = r.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
             AND o.order_ts >= now() - INTERVAL '60 days' -- 선정 고객의 최근 60일 매출
       ), 2) AS revenue_60d
FROM ranked r
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01) -- 전체 고객의 상위 1%, 정확히 30명
ORDER BY r.lifetime_rank;
```

```
WITH lifetime_sales AS (                            -- [②] 실제 상위 30명 결과 조회
    SELECT c.customer_id, c.full_name,
           (
               SELECT COALESCE(SUM(oi.line_total), 0)
               FROM ecom.orders o
               JOIN ecom.order_items oi ON oi.order_id = o.order_id
               WHERE o.customer_id = c.customer_id
                 AND o.order_status IN ('paid', 'shipped', 'delivered')
           ) AS lifetime_revenue
    FROM ecom.customers c
), ranked AS (
    SELECT customer_id, full_name, lifetime_revenue,
           ROW_NUMBER() OVER (
               ORDER BY lifetime_revenue DESC, customer_id
           ) AS lifetime_rank,
           COUNT(*) OVER () AS customer_count
    FROM lifetime_sales
)
SELECT r.customer_id, r.full_name, r.lifetime_rank,
       ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,
       ROUND((
           SELECT COALESCE(SUM(oi.line_total), 0)
           FROM ecom.orders o
           JOIN ecom.order_items oi ON oi.order_id = o.order_id
           WHERE o.customer_id = r.customer_id
             AND o.order_status IN ('paid', 'shipped', 'delivered')
             AND o.order_ts >= now() - INTERVAL '60 days'
       ), 2) AS revenue_60d
FROM ranked r
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01)
ORDER BY r.lifetime_rank;
```

```
QUERY PLAN
Sort  (cost=167358.09..167360.59 rows=1000 width=93) (actual time=20.166..20.171 rows=30 loops=1)
  Sort Key: r.lifetime_rank
  Sort Method: quicksort  Memory: 27kB
  Buffers: shared hit=21777
  ->  Subquery Scan on r  (cost=100456.54..167308.26 rows=1000 width=93) (actual time=17.001..19.786 rows=30 loops=1)
        Filter: ((r.lifetime_rank)::numeric <= ceil(((r.customer_count)::numeric * 0.01)))
        Rows Removed by Filter: 2970
        Buffers: shared hit=21777
        ->  WindowAgg  (cost=100456.54..150539.51 rows=3000 width=69) (actual time=16.866..17.287 rows=3000 loops=1)
              Buffers: shared hit=19085
              ->  WindowAgg  (cost=50309.53..100435.76 rows=3000 width=61) (actual time=15.418..16.336 rows=3000 loops=1)
                    Buffers: shared hit=19085
                    ->  Sort  (cost=50309.51..50317.01 rows=3000 width=53) (actual time=15.403..15.550 rows=3000 loops=1)
                          Sort Key: ((SubPlan 2)) DESC, c.customer_id
                          Sort Method: quicksort  Memory: 235kB
                          Buffers: shared hit=19085
                          ->  Seq Scan on customers c  (cost=0.00..50136.25 rows=3000 width=53) (actual time=0.260..13.886 rows=3000 loops=1)
                                Buffers: shared hit=19085
                                SubPlan 2
                                  ->  Aggregate  (cost=16.68..16.69 rows=1 width=32) (actual time=0.004..0.004 rows=1 loops=3000)
                                        Buffers: shared hit=19045
                                        ->  Nested Loop  (cost=0.57..16.67 rows=3 width=6) (actual time=0.002..0.003 rows=3 loops=3000)
                                              Buffers: shared hit=19045
                                              ->  Index Scan using idx_orders_customer_ts on orders o_1  (cost=0.28..8.30 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=3000)
                                                    Index Cond: (customer_id = c.customer_id)
                                                    Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                                    Rows Removed by Filter: 0
                                                    Buffers: shared hit=9787
                                              ->  Index Scan using idx_order_items_order on order_items oi_1  (cost=0.29..8.34 rows=3 width=14) (actual time=0.001..0.001 rows=3 loops=3068)
                                                    Index Cond: (order_id = o_1.order_id)
                                                    Buffers: shared hit=9258
```

```
      SubPlan 1
        ->  Aggregate  (cost=16.69..16.70 rows=1 width=32) (actual time=0.059..0.059 rows=1 loops=30)
              Buffers: shared hit=2692
              ->  Nested Loop  (cost=0.57..16.68 rows=3 width=6) (actual time=0.005..0.053 rows=63 loops=30)
                    Buffers: shared hit=2692
                    ->  Index Scan using idx_orders_customer_ts on orders o  (cost=0.28..8.31 rows=1 width=8) (actual time=0.003..0.012 rows=22 loops=30)
                          Index Cond: ((customer_id = r.customer_id) AND (order_ts >= (now() - '60 days'::interval)))
                          Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                          Rows Removed by Filter: 0
                          Buffers: shared hit=663
                    ->  Index Scan using idx_order_items_order on order_items oi  (cost=0.29..8.34 rows=3 width=14) (actual time=0.001..0.001 rows=3 loops=672)
                          Index Cond: (order_id = o.order_id)
                          Buffers: shared hit=2029
Planning:
  Buffers: shared hit=27
Planning Time: 1.756 ms
Execution Time: 20.357 ms
```

customers 1

WITH lifetime_sales AS (SELECT c.customer_id, c.full_nai | Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	full_name	lifetime_rank	lifetime_revenue	revenue_60d
1	8	Customer 8	1	35,154.22	32,794.66
2	16	Customer 16	2	31,003.24	31,003.24
3	18	Customer 18	3	30,283.04	29,772.64
4	14	Customer 14	4	29,322.59	29,322.59
5	6	Customer 6	5	28,703.23	28,703.23
6	13	Customer 13	6	28,633.41	28,633.41
7	2	Customer 2	7	28,623.31	28,623.31
8	1	Customer 1	8	28,546.26	28,546.26
9	20	Customer 20	9	28,531.13	28,531.13
10	9	Customer 9	10	28,488.91	28,488.91
11	17	Customer 17	11	27,317.53	26,943.11
12	24	Customer 24	12	27,075.74	27,075.74
13	23	Customer 23	13	25,717.57	25,717.57
14	30	Customer 30	14	24,815.39	24,815.39
15	4	Customer 4	15	24,726.24	24,101.82
16	12	Customer 12	16	22,419.91	21,470.83
17	19	Customer 19	17	22,412.27	22,412.27
18	22	Customer 22	18	22,206.5	21,128.93
19	7	Customer 7	19	22,034.39	22,034.39
20	29	Customer 29	20	21,696.95	21,696.95
21	11	Customer 11	21	21,635.89	20,199.13
22	5	Customer 5	22	21,557.53	21,557.53
23	15	Customer 15	23	21,466.9	21,466.9
24	21	Customer 21	24	21,430.97	21,430.97
25	28	Customer 28	25	20,490.17	19,053.41
26	26	Customer 26	26	18,974.16	18,974.16
27	3	Customer 3	27	18,431.44	18,220.72
28	10	Customer 10	28	17,990.28	17,990.28
29	27	Customer 27	29	17,135.65	17,135.65
30	25	Customer 25	30	15,993.86	15,993.86

느린 이유와 개선 방법

튜닝 전에는 고객 3,000명의 누적매출과 상위 고객의 최근 매출을 상관 서브쿼리로 반복 계산한다. 튜닝 후에는 전체 고객을 기준으로 누적매출과 최근 60일 매출을 한 번에 집계하고, `ROW_NUMBER()`와 전체 고객 수를 이용해 30명을 정확히 추출한다.

Q10 튜닝 후

```sql
-- Q10. 전체 3,000명의 누적매출과 최근 60일 매출을 한 번에 집계한 뒤
--      누적매출 상위 1%(30명)를 정확히 선정
-- 튜닝: 누적매출과 60일 매출을 한 번의 GROUP BY로 동시에 계산한다.
-- [① 실행계획 확인용] 고객별 SubPlan 없이 한 번에 집계하는지 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH customer_sales AS (
    SELECT c.customer_id,
           c.full_name,
           COALESCE(SUM(oi.line_total), 0) AS lifetime_revenue, -- 고객 누적매출
           COALESCE(SUM(oi.line_total) FILTER (
               WHERE o.order_ts >= now() - INTERVAL '60 days'
           ), 0) AS revenue_60d                      -- 같은 집계에서 최근 60일 매출만 합산
    FROM ecom.customers c
    LEFT JOIN ecom.orders o                          -- 주문 없는 고객도 전체 3,000명에 포함
      ON o.customer_id = c.customer_id
     AND o.order_status IN ('paid', 'shipped', 'delivered')
    LEFT JOIN ecom.order_items oi ON oi.order_id = o.order_id
    GROUP BY c.customer_id, c.full_name
), ranked AS (
    SELECT customer_id, full_name, lifetime_revenue, revenue_60d,
           ROW_NUMBER() OVER (                       -- 누적매출 순서대로 중복 없는 순번 부여
               ORDER BY lifetime_revenue DESC, customer_id
           ) AS lifetime_rank,
           COUNT(*) OVER () AS customer_count        -- 전체 고객 수를 각 행에 표시
    FROM customer_sales
)
SELECT customer_id, full_name, lifetime_rank,
       ROUND(lifetime_revenue, 2) AS lifetime_revenue,
       ROUND(revenue_60d, 2) AS revenue_60d
FROM ranked
WHERE lifetime_rank <= CEIL(customer_count * 0.01) -- 전체 고객 중 상위 1%, 정확히 30명
ORDER BY lifetime_rank;
```

```sql
WITH customer_sales AS (                             -- [②] 실제 상위 30명 결과 조회
    SELECT c.customer_id, c.full_name,
           COALESCE(SUM(oi.line_total), 0) AS lifetime_revenue,
           COALESCE(SUM(oi.line_total) FILTER (
               WHERE o.order_ts >= now() - INTERVAL '60 days'
           ), 0) AS revenue_60d
    FROM ecom.customers c
    LEFT JOIN ecom.orders o
      ON o.customer_id = c.customer_id
     AND o.order_status IN ('paid', 'shipped', 'delivered')
    LEFT JOIN ecom.order_items oi ON oi.order_id = o.order_id
    GROUP BY c.customer_id, c.full_name
), ranked AS (
    SELECT customer_id, full_name, lifetime_revenue, revenue_60d,
           ROW_NUMBER() OVER (
               ORDER BY lifetime_revenue DESC, customer_id
           ) AS lifetime_rank,
           COUNT(*) OVER () AS customer_count
    FROM customer_sales
)
SELECT customer_id, full_name, lifetime_rank,
       ROUND(lifetime_revenue, 2) AS lifetime_revenue,
       ROUND(revenue_60d, 2) AS revenue_60d
FROM ranked
WHERE lifetime_rank <= CEIL(customer_count * 0.01)
ORDER BY lifetime_rank;
```

Results 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH customer_sales AS (SE | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Sort  (cost=1053.20..1055.70 rows=1000 width=93) (actual time=24.606..24.612 rows=30 loops=1)
  Sort Key: ranked.lifetime_rank
  Sort Method: quicksort  Memory: 27kB
  Buffers: shared hit=180
  ->  Subquery Scan on ranked  (cost=930.83..1003.37 rows=1000 width=93) (actual time=23.249..24.592 rows=30 loops=1)
        Filter: ((ranked.lifetime_rank)::numeric <= ceil(((ranked.customer_count)::numeric * 0.01)))
        Rows Removed by Filter: 2970
        Buffers: shared hit=180
        ->  WindowAgg  (cost=930.83..930.87 rows=3000 width=101) (actual time=23.243..23.748 rows=3000 loops=1)
              Buffers: shared hit=180
              ->  WindowAgg  (cost=833.39..893.37 rows=3000 width=93) (actual time=21.834..22.644 rows=3000 loops=1)
                    Buffers: shared hit=180
                    ->  Sort  (cost=833.37..840.87 rows=3000 width=85) (actual time=21.828..21.957 rows=3000 loops=1)
                          Sort Key: customer_sales.lifetime_revenue DESC, customer_sales.customer_id
                          Sort Method: quicksort  Memory: 246kB
                          Buffers: shared hit=180
                          ->  Subquery Scan on customer_sales  (cost=615.11..660.11 rows=3000 width=85) (actual time=18.510..20.082 rows=3000 loops=1)
                                Buffers: shared hit=180
                                ->  HashAggregate  (cost=615.11..660.11 rows=3000 width=85) (actual time=18.510..19.743 rows=3000 loops=1)
                                      Group Key: c.customer_id
                                      Batches: 1  Memory Usage: 1905kB
                                      Buffers: shared hit=180
                                      ->  Hash Right Join  (cost=238.93..491.51 rows=8240 width=35) (actual time=4.543..11.788 rows=9013 loops=1)
                                            Hash Cond: (o.customer_id = c.customer_id)
                                            Buffers: shared hit=180
                                            ->  Hash Right Join  (cost=131.43..362.34 rows=8240 width=22) (actual time=2.665..7.421 rows=8230 loops=1)
                                                  Hash Cond: (oi.order_id = o.order_id)
                                                  Buffers: shared hit=140
                                                  ->  Seq Scan on order_items oi  (cost=0.00..203.67 rows=10367 width=14) (actual time=0.015..1.508 rows=10367 loops
                                                        Buffers: shared hit=100
                                                  ->  Hash  (cost=93.08..93.08 rows=3068 width=24) (actual time=2.616..2.617 rows=3068 loops=1)
                                                        Buckets: 4096  Batches: 1  Memory Usage: 200kB
                                                        Buffers: shared hit=40
                                                        ->  Seq Scan on orders o  (cost=0.00..93.08 rows=3068 width=24) (actual time=0.019..1.762 rows=3068 loops=1)
                                                              Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
                                                              Rows Removed by Filter: 792
                                                              Buffers: shared hit=40
                                            ->  Hash  (cost=70.00..70.00 rows=3000 width=21) (actual time=1.853..1.853 rows=3000 loops=1)
                                                  Buckets: 4096  Batches: 1  Memory Usage: 190kB
                                                  Buffers: shared hit=40
                                                  ->  Seq Scan on customers c  (cost=0.00..70.00 rows=3000 width=21) (actual time=0.028..0.932 rows=3000 loops=1)
                                                        Buffers: shared hit=40
```

customers 1

WITH customer_sales AS (SELECT c.customer_id, c.full_name, COALESC | Enter a SQL expression to filter results (use Ctrl+Space)

	customer_id	full_name	lifetime_rank	lifetime_revenue	revenue_60d
1	8	Customer 8	1	35,154.22	32,794.66
2	16	Customer 16	2	31,003.24	31,003.24
3	18	Customer 18	3	30,283.04	29,772.64
4	14	Customer 14	4	29,322.59	29,322.59
5	6	Customer 6	5	28,703.23	28,703.23
6	13	Customer 13	6	28,633.41	28,633.41
7	2	Customer 2	7	28,623.31	28,623.31
8	1	Customer 1	8	28,546.26	28,546.26
9	20	Customer 20	9	28,531.13	28,531.13
10	9	Customer 9	10	28,488.91	28,488.91
11	17	Customer 17	11	27,317.53	26,943.11
12	24	Customer 24	12	27,075.74	27,075.74
13	23	Customer 23	13	25,717.57	25,717.57
14	30	Customer 30	14	24,815.39	24,815.39
15	4	Customer 4	15	24,726.24	24,101.82
16	12	Customer 12	16	22,419.91	21,470.83
17	19	Customer 19	17	22,412.27	22,412.27
18	22	Customer 22	18	22,206.5	21,128.93
19	7	Customer 7	19	22,034.39	22,034.39
20	29	Customer 29	20	21,696.95	21,696.95
21	11	Customer 11	21	21,635.89	20,199.13
22	5	Customer 5	22	21,557.53	21,557.53
23	15	Customer 15	23	21,466.9	21,466.9
24	21	Customer 21	24	21,430.97	21,430.97
25	28	Customer 28	25	20,490.17	19,053.41
26	26	Customer 26	26	18,974.16	18,974.16
27	3	Customer 3	27	18,431.44	18,220.72
28	10	Customer 10	28	17,990.28	17,990.28
29	27	Customer 27	29	17,135.65	17,135.65
30	25	Customer 25	30	15,993.86	15,993.86

```
Planning:
  Buffers: shared hit=33
Planning Time: 4.017 ms
Execution Time: 24.904 ms
```

검증 항목	기대값	실제값
전체 고객 수	3,000	_____
상위 1% 고객 수	30	_____
누적매출 1위 고객	-	_____
해당 고객 최근 60일 매출	-	_____

구분	반복 SubPlan	Buffers	Execution Time
튜닝 전	_____	_____	_____ ms
튜닝 후	_____	_____	_____ ms

결과 해석: _____

13. Q11 - 안전 나눗셈 함수 비교

함수 정의 차이

함수	정의 파일	분모가 0일 때
`ecom.f_safe_div`	스키마 파일	`0` 반환
`ecom.safe_div`	시드 파일	`NULL` 반환

Q11 함수 비교

```sql
-- Q11. 스키마와 시드에 정의된 두 함수의 정상 분모 결과 비교
-- [① 정상 분모] 분모가 0이 아니면 두 함수가 같은 계산 결과를 반환
SELECT ecom.f_safe_div(
            SUM(oi.line_total), COUNT(DISTINCT o.order_id)
        ) AS schema_f_safe_div_aov,
        ecom.safe_div(
            SUM(oi.line_total), COUNT(DISTINCT o.order_id)
        ) AS seed_safe_div_aov
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered');
```

orders(+) 1

SELECT ecom.f_safe_div(SUM(oi.line_total), COUNT(DISTINCT o.order_i

	123 schema_f_safe_div_aov	123 seed_safe_div_aov
1	886.2541297262	886.2541297262

```sql
-- 차이점: f_safe_div는 0, safe_div는 NULL을 반환한다.
-- [② 0 분모] 0 반환과 NULL 반환의 업무 정책 차이 확인
SELECT ecom.f_safe_div(100, 0) AS schema_function_returns_zero, -- 0으로 나누면 숫자 0
       ecom.safe_div(100, 0) AS seed_function_returns_null;     -- 0으로 나누면 NULL(값 없음)
```

sults 1 | Statistics 1

LECT ecom.f_safe_div(100, 0) AS schema_function_returns_zero, econ

123 schema_function_returns_zero	123 seed_function_returns_null
0	[NULL]

실행 결과

테스트	`f_safe_div`	`safe_div`	해석
정상 AOV	_____	_____	분모가 0이 아니면 같은 계산 결과
`100 / 0`	`0`	`NULL`	보고서 요구에 맞는 정책을 선택해야 함

14. 물리 조인 3종 비교

실행 결과 화면

Nested Loop Join

Nested Loop Join

Hash Join

Hash Join

Merge Join

Merge Join

구분	처리 방식	일반적으로 유리한 조건	Buffers	Execution Time
Nested Loop Join	외부 행마다 내부 입력 탐색	외부 입력이 작고 내부 인덱스가 있을 때	_____	_____ ms
Hash Join	작은 입력 해시 생성 후 큰 입력 탐색	큰 비정렬 동등 조인	_____	_____ ms

구분	처리 방식	일반적으로 유리한 조건	Buffers	Execution Time
Merge Join	정렬된 두 입력을 순차 병합	양쪽이 이미 정렬된 큰 입력	_____	_____ ms

본인 환경에서의 결론

15. Materialized View 활용

생성 목적

매일 주문과 주문상품을 다시 조인·집계하지 않고, 미리 계산한 일별 주문 수와 GMV를 빠르게 조회한다.

생성 및 결과 화면

MV 생성

MV 생성

원본 쿼리 계획

원본 쿼리 계획

MV 조회 결과

MV 조회 결과

MV 갱신

MV 갱신

구분	핵심 실행계획	Buffers	Execution Time
원본 JOIN + GROUP BY	_____	_____	_____ ms
`mv_daily_gmv` 조회	_____	_____	_____ ms

갱신 전략

- 최초 적재: `REFRESH MATERIALIZED VIEW ecom.mv_daily_gmv;`
- 운영 갱신: `REFRESH MATERIALIZED VIEW CONCURRENTLY ecom.mv_daily_gmv;`
- 제안 주기: 매일 오후 3시
- 주의점: 오후 3시 이후 변경사항은 다음 갱신 전까지 조회 결과에 반영되지 않는다.

16. DBMS별 옵티마이저 비교 (Option)

자세한 조사 내용은 `종합실습4_06_DBMS별_옵티마이저_비교.md`를 참고한다.

DBMS	대표 조인	실행계획 확인	특징
PostgreSQL	Nested Loop, Hash, Merge	`EXPLAIN ANALYZE`	통계·비용 기반, 확장 통계, GEQO
MySQL	Nested Loop/BKA, Hash	`EXPLAIN ANALYZE`	optimizer switch·힌트·히스토그램
Oracle	Nested Loop, Hash, Sort Merge	`DBMS_XPLAN`	Adaptive Plan, SQL Plan Management
SQL Server	Nested Loops, Hash, Merge, Adaptive	Actual Execution Plan	Query Store, Intelligent Query Processing

17. 전체 튜닝 전후 비교

문항	튜닝 전 방식	튜닝 후 방식	전 Buffers	후 Buffers	전 시간	후 시간	감소율
Q1	함수 조건 + 상관 집계	SARG 범위 + 단일 집계	_____	_____	_____	_____	_____%
Q2	상세행 + DISTINCT	주문별 선집계	_____	_____	_____	_____	_____%
Q3	비-SARG 기간 조건	최근 주문 선필터	_____	_____	_____	_____	_____%
Q4	제품별 상관 서브쿼리	전체 1회 집계 + RANK	_____	_____	_____	_____	_____%
Q5	고객별 3회 반복 조회	주문 선집계 + 고객 GROUP BY	_____	_____	_____	_____	_____%
Q6	고객별 EXISTS	Window + BOOL_OR	_____	_____	_____	_____	_____%
Q7	임계치 상관 조회	직접 비교 + 부분 인덱스	_____	_____	_____	_____	_____%
Q8	AVG/COUNT 반복	GROUP BY + HAVING	_____	_____	_____	_____	_____%

문항	튜닝 전 방식	튜닝 후 방식	전 Buffers	후 Buffers	전 시간	후 시간	감소율
Q9	상세행 + DISTINCT	주문별 선집계 + AVG	_____	_____	_____	_____	_____%
Q10	누적/60일 매출 반복 조회	전체 1회 집계 + 정확히 30명	_____	_____	_____	_____	_____%

감소율 계산식:

```
(튜닝 전 시간 - 튜닝 후 시간) / 튜닝 전 시간 × 100
```

18. 최종 분석

요청 대응 관점

어떤 주문 상태와 기간을 포함하는지, 주문 수와 주문상품 수를 혼동하지 않았는지, 상위 고객의 선정 기준과 표시 기간이 다른지 먼저 정의해야 한다. 특히 Q10은 최근 60일 매출로 상위 고객을 선정하는 것이 아니라 누적매출로 30명을 선정한 뒤 최근 60일 매출을 계산한다.

튜닝 관점

한계와 추가 확인사항

- 랜덤 시드이므로 재실행 시 결과가 달라진다.
- 로컬 단일 사용자 환경의 실행시간은 운영 부하를 반영하지 않는다.
- 쿠폰 사용 여부와 AOV의 관계는 인과관계가 아니다.
- 인덱스는 조회 성능을 높이지만 INSERT/UPDATE와 저장공간 비용을 추가한다.

최종 결론